

Bada

The Unknown-Prior Engine Language

[Complete source listing](#) - [lexer](#), [parser](#), [interpreter](#), [Bada->C compiler](#)

Contents

README.md	Project README
Makefile	Build file
src/bada.h	Shared header: tokens, AST, value model, API
src/lexer.c	Tokenizer
src/ast.c	AST node construction
src/parser.c	Recursive-descent parser
src/interp.c	Tree-walking interpreter + engine builtins
src/transpile.c	Bada -> C compiler (transpiler)
src/main.c	CLI driver
examples/hello.bada	Example: language sanity check
examples/engine.bada	Example: full Unknown-Prior Engine
examples/core.bada	Example: engine core for the compiler path

Build: `make -> build/bada`

Run: `build/bada run examples/engine.bada` (interpreter)

Compile: `build/bada build examples/core.bada -o p` (Bada -> C -> native)

Verified: reproduces the manuscript's verbatim output -- $V=8$, tokens 3 and 6 ruled out ($p=0$), zero preservation $[3,6] \rightarrow [3,6]$, property $|\text{psil}|^2 == a$ with max deviation $5.55e-17$, and strictly decreasing unknown-prior entropy.

README.md

Project README

```

1 # Bada — The Unknown-Prior Engine Language
2
3 Bada is a small, object-oriented programming language specialised for
4 generative-AI inference, built directly around the Unknown-Prior Engine
5 described in the Yamaguchi ultranetwork manuscript (`bada_engine_full_source.pdf`).
6
7 The manuscript's own note is that "Bada is a specification notation, not an
8 executable language." This project takes the opposite step: it makes Bada
9 executable, with a real lexer, parser, tree-walking interpreter, and a
10 Bada → C compiler (transpiler) — all written in C.
11
12 The engine's two provable properties are reproduced exactly by both the
13 interpreter and the compiled binary:
14
15 1. Zero preservation — confident zeros (ruled-out tokens) survive untouched.
16 2.  $|\psi|^2 = a$  — the unit-modulus phase only reweights, never destroys
17    (max deviation `5.55e-17`, machine precision).
18
19 ---
20
21 ## Build
22
23 ```sh
24 make
25 ```
26
27 Produces `build/bada`. Requires only `gcc` and `libm` (uses C11 `<complex.h`).
28
29 ## Usage
30
31 ```sh
32 build/bada run    examples/engine.bada    # interpret
33 build/bada build  examples/core.bada -o p  # compile Bada -> C -> native binary
34 build/bada emit   examples/core.bada -o p.c # emit C only
35 build/bada tokens examples/engine.bada     # dump token stream
36 build/bada ast    examples/engine.bada     # dump AST
37 ```
38
39 ### Run the full engine (interpreter)
40
41 ```sh
42 build/bada run examples/engine.bada
43 ```
44
45 reproduces the manuscript's verbatim §9 output: `base zeros = [3, 6]`,
46 `posterior zeros = [3, 6]`, property (ii) `5.55e-17`, and the strictly
47 decreasing unknown-prior entropy `2.0794 → 1.9733 → 1.8364`.
48
49 ### Compile the core to a native binary
50
51 ```sh
52 build/bada build examples/core.bada -o build/core
53 build/core
54 ```
55
56 The compiled C produces `base a == q_hat` to machine precision — the same
57 invariant, now in a standalone executable.
58
59 ---
60
61 ## The Bada language
62
63 ### Native types
64
65 | Type          | Meaning |
66 |-----|-----|
67 | `num`      | IEEE double |
68 | `str`, `bool`, `nil` | scalars |
69 | `array` `[...] | ordered list |
70 | `dist`      | native probability vector (auto-normalised, max-entropy aware) |
71 | `phase`    | native complex unit-modulus vector (Euler `e^{i\theta}`) |
72 | `tuplespace` | the append-only immutable ledger (`\Omega` / `tuplespace`) |
73 | `struct` / instance | object templates and instances |
74 | `func` / lambda | closures; lambdas written `|x| body` |
75
76 ### Manuscript operators (faithful)
77
78 | Op      | Name          | Semantics |
79 |-----|-----|-----|

```

```

80 | `:=` | bind | introduce a binding |
81 | `>>` | commit | append a fact to the append-only ledger, then yield RHS |
82 | `=>` / `->` | query | pattern query / map result |
83 | `:=>` | def-arrow | definitional arrow (asperl / function body) |
84 | `<-` | append | append an element to an array field |
85 | `~` | consistency | gradual-type consistency (`STAR ~ x` is always true) |
86 | `::` | scope | namespace resolution (`Omega::DATABASE`) |
87 | <code>#124;x#124;</code> | pipe params | lambda / method parameter list |
88 | `@reviser { ... }` | learning loop | reflective rewrite block |
89
90 ### Engine builtins (native)
91
92 `softmax`, `entropy`, `unknown_prior(V)`, `update(prior, evidence [,lr])`,
93 `manifold_embed(x [,W])`, `cognitive_system(base, modules, betas)`,
94 `dist(arr)`, `zeros_of(d)`, `maxdiff(a,b)`, `last_a()`, `len`, `sqrt`, `log`,
95 `exp`, `abs`, `f5`, `sci`, `ledger`.
96
97 ### The core: entropy-phase cognitive system
98
99 With `a = softmax(z)`, modules `q_m` of entropy `H_m` and phase `phi_m`, gains `beta_m`:
100
101 ...
102  $\psi_i = \sqrt{a_i \cdot \exp(i \cdot \sum_m \beta_m H_m \phi_m(i))}$ 
103  $q_i = |\psi_i|^2 / \sum_j |\psi_j|^2$ 
104 ...
105
106 Because  $|e^{i\theta}| = 1$ ,  $|\psi_i|^2 = a_i$  exactly: confident zeros are
107 preserved and uninformative (constant-`H`) modules cancel. The engine starts
108 in honest ignorance (`unknown_prior` = uniform = max entropy, Jaynes) and
109 earns certainty from the append-only ledger.
110
111 ---
112
113 ## Two registers, one behaviour
114
115 - Interpreter (`src/interp.c`): full Bada – structs, `@reviser`, `Omega::`
116 blocks, `each`, gradual typing, the ledger, and all engine builtins.
117 - Compiler (`src/transpile.c`): lowers the engine-shaped subset to a
118 self-contained C program (embedded runtime + `b_cognitive`, `b_softmax`, ...),
119 then invokes `gcc`. The compiled binary matches the interpreter's numerics.
120
121 Where the two ever disagree, the manuscript's rule applies: the executable is
122 authoritative.
123
124 ## Files
125
126 ...
127 src/bada.h shared defs: tokens, AST, value model, API
128 src/lexer.c tokenizer (all manuscript operators: >> := => <- ~ :: [...])
129 src/ast.c AST node construction
130 src/parser.c recursive-descent parser (Omega::, asperl, @reviser, def, ...)
131 src/interp.c tree-walking interpreter + native engine builtins
132 src/transpile.c Bada -> C compiler with embedded runtime
133 src/main.c the `bada` CLI driver
134 examples/hello.bada language sanity check
135 examples/engine.bada full Unknown-Prior Engine (Modules 1-7), interpreter
136 examples/core.bada engine core for the compiler path
137 ...
138
139 ## Scope / honesty
140
141 This implements the legitimate shapes of the manuscript only: maximum-entropy
142 priors, the entropic summand  $-p \log p$ , and Euler's unit-modulus phase
143  $e^{i\theta}$ . It makes no claim about the Riemann Hypothesis or about  $\zeta/\Gamma$ ; the
144 manuscript's invalid identities are not used. The one mathematical fact the
145 design relies on – a unit-modulus factor preserves zeros – is verified directly
146 by the run ( $5.55e-17$ ).

```

Makefile

Build file

```
1 CC      = gcc
2 CFLAGS  = -std=c11 -O2 -Wall -Wextra -Wno-unused-parameter
3 LDFLAGS = -lm
4 SRC     = src/lexer.c src/ast.c src/parser.c src/interp.c src/transpile.c src/main.c
5 OBJ     = $(SRC:.c=.o)
6 BIN     = build/bada
7
8 all: $(BIN)
9
10 $(BIN): $(SRC) src/bada.h
11     @mkdir -p build
12     $(CC) $(CFLAGS) -o $(BIN) $(SRC) $(LDFLAGS)
13
14 clean:
15     rm -f $(BIN) src/*.o *.gen.c a.out
16
17 .PHONY: all clean
```

src/bada.h

Shared header: tokens, AST, value model, API

```

1  /* bada.h -- The Bada language: shared definitions.
2  *
3  * Bada is a generative-AI-oriented, object-oriented language built around the
4  * Unknown-Prior Engine. Native types: dist (max-entropy probability vectors),
5  * phase (unit-modulus complex factors), plus an append-only TupleSpace ledger.
6  *
7  * This header is shared by the lexer, parser, tree-walking interpreter, and the
8  * C transpiler.
9  */
10 #ifndef BADA_H
11 #define BADA_H
12
13 #define _GNU_SOURCE
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17 #include <stdbool.h>
18 #include <math.h>
19 #include <complex.h>
20 #include <stdarg.h>
21
22 /* -----
23  * Tokens
24  * ----- */
25 typedef enum {
26     /* literals & identifiers */
27     T_INT, T_FLOAT, T_STRING, T_IDENT,
28     /* keywords */
29     T_OMEGA,      /* Omega -- the root namespace / engine object */
30     T_DATABASE,   /* DATABASE */
31     T_TUPLESPEC,  /* tuplespace / TupleSpace */
32     T_ASPEARAL,   /* asperal -- gradual (semi-static) typed declaration */
33     T_STRUCT,     /* struct -- object/record body */
34     T_DEF,        /* def -- method / function */
35     T_TYPEDEF,    /* typedef */
36     T_RETURN,     /* return */
37     T_IF, T_ELSE, T_WHILE, T_FOR, T_IN,
38     T_REVISER,    /* @reviser block (reflective learning loop) */
39     T_STATIC, T_DYNAMIC, T_REFINE, /* asperal type kinds */
40     T_STAR,       /* star -- the dynamic type */
41     T_TRUE, T_FALSE, T_NIL,
42     T_LET,        /* let / := binding */
43     T_PRINT,      /* print */
44     T_EACH,       /* each -- iteration combinator */
45     /* native engine builtins are ordinary idents resolved at runtime */
46     /* operators */
47     T_ASSIGN,     /* = */
48     T_WALRUS,     /* := -- bind */
49     T_COMMIT,     /* >> -- commit / append to ledger */
50     T_ARROW,      /* => -- query / map result */
51     T_QUERY,      /* -> -- alternate query/map arrow */
52     T_GET,        /* reserved (member-access marker, unused in lexer) */
53     T_DEFARROW,   /* :=> -- definitional arrow (asperal) */
54     T_LARROW,     /* <- -- append element */
55     T_CONSIST,    /* ~ -- gradual consistency */
56     T_SCOPE,      /* :: -- namespace resolution */
57     T_PLUS, T_MINUS, T_STAR_OP, T_SLASH, T_PERCENT,
58     T_EQ, T_NE, T_LT, T_LE, T_GT, T_GE,
59     T_AND, T_OR, T_NOT,
60     T_DOT, T_COMMA, T_COLON, T_SEMI, T_PIPE, /* | for |params| and union */
61     T_LPAREN, T_RPAREN, T_LBRACE, T_RBRACE, T_LBRACK, T_RBRACK,
62     T_EOF, T_ERROR
63 } TokType;
64
65 typedef struct {
66     TokType type;
67     char *lexeme; /* owned copy */
68     double num; /* for T_INT / T_FLOAT */
69     int line;
70 } Token;
71
72 /* -----
73  * AST
74  * ----- */
75 typedef enum {
76     N_INT, N_FLOAT, N_STRING, N_BOOL, N_NIL, N_IDENT,
77     N_BINARY, N_UNARY, N_ASSIGN, N_BIND, /* := */
78     N_CALL, N_INDEX, N_GET, N_ARRAY, N_BLOCK,
79     N_IF, N_WHILE, N_FOR, N_RETURN, N_PRINT,

```

```

80     N_FUNC, N_STRUCT, N_FIELD, N_METHOD,
81     N_OMEGA_DB,      /* Omega::DATABASE[tuplespace] { ... } */
82     N_REVISER,       /* @reviser block */
83     N_COMMIT,        /* x >> y (commit/append) */
84     N_QUERY,         /* x => y (query/map) */
85     N_LARROW,        /* facts <- fact */
86     N_CONSIST,       /* s ~ t */
87     N_ASPELAL,       /* asperal Name :struct {...} or asperal x :=> ... */
88     N_PIPEPARAMS,    /* [a,b] parameter list (lambda head) */
89     N_LAMBDA,        /* [params] body */
90     N_EACH,          /* coll.each |x| do ... end */
91     N_PROGRAM
92 } NodeType;
93
94 typedef struct Node Node;
95 struct Node {
96     NodeType type;
97     int line;
98     /* literal */
99     double num;
100    char *str;      /* string / ident name / operator text */
101    bool boolean;
102    /* children */
103    Node **kids;
104    int nkids;
105    int cap;
106    /* function/struct/lambda */
107    char **params;
108    int nparams;
109    Node *body;
110 };
111
112 Node *node_new(NodeType t, int line);
113 void node_push(Node *parent, Node *child);
114 void node_free(Node *n);
115
116 /* lexer */
117 typedef struct {
118     const char *src;
119     int pos, line;
120 } Lexer;
121 void lexer_init(Lexer *lx, const char *src);
122 Token lexer_next(Lexer *lx);
123 void token_free(Token *t);
124 const char *toktype_name(TokType t);
125
126 /* parser */
127 Node *parse_program(const char *src);
128
129 /* interpreter */
130 int interp_run(Node *program);
131
132 /* transpiler (Bada -> C) */
133 int transpile_to_c(Node *program, const char *out_path);
134
135 #endif /* BADA_H */

```

src/lexer.c

Tokenizer

```

1 #define _GNU_SOURCE
2 /* lexer.c -- tokenizer for Bada. */
3 #include "bada.h"
4 #include <ctype.h>
5
6 static const struct { const char *kw; TokType t; } KEYWORDS[] = {
7     {"Omega", T_OMEGA}, {"DATABASE", T_DATABASE},
8     {"tuplespace", T_TUPLESPEACE}, {"TupleSpace", T_TUPLESPEACE},
9     {"asperial", T_ASPIRAL}, {"struct", T_STRUCT},
10    {"def", T_DEF}, {"typedef", T_TYPEDEF},
11    {"return", T_RETURN}, {"if", T_IF},
12    {"else", T_ELSE}, {"while", T_WHILE},
13    {"for", T_FOR}, {"in", T_IN},
14    {"static", T_STATIC}, {"dynamic", T_DYNAMIC},
15    {"refine", T_REFINE}, {"star", T_STAR},
16    {"true", T_TRUE}, {"false", T_FALSE},
17    {"nil", T_NIL}, {"let", T_LET},
18    {"print", T_PRINT}, {"each", T_EACH},
19    {"do", T_ERROR /*soft*/}, {NULL, T_EOF}
20 };
21
22 void lexer_init(Lexer *lx, const char *src){ lx->src=src; lx->pos=0; lx->line=1; }
23
24 static char peek(Lexer*lx){ return lx->src[lx->pos]; }
25 static char peek2(Lexer*lx){ return lx->src[lx->pos]?lx->src[lx->pos+1]:'\0'; }
26 static char adv(Lexer*lx){ char c=lx->src[lx->pos++]; if(c=='\n')lx->line++; return c; }
27
28 static Token mk(TokType t,const char*s,int line){
29     Token tk; tk.type=t; tk.lexeme=s?strdup(s):NULL; tk.num=0; tk.line=line; return tk;
30 }
31
32 static void skip_ws(Lexer*lx){
33     for(;;){
34         char c=peek(lx);
35         if(c==' '||c=='\t'||c=='\r'||c=='\n'){ adv(lx); }
36         else if(c=='#'){ while(peek(lx)&&peek(lx)!='\n') adv(lx); }
37         else if(c=='/'&&peek2(lx)=='/'){ while(peek(lx)&&peek(lx)!='\n') adv(lx); }
38         else break;
39     }
40 }
41
42 void token_free(Token*t){ if(t->lexeme){ free(t->lexeme); t->lexeme=NULL; } }
43
44 Token lexer_next(Lexer*lx){
45     skip_ws(lx);
46     int line=lx->line;
47     char c=peek(lx);
48     if(c=='\0') return mk(T_EOF,"<eof>",line);
49
50     /* numbers */
51     if(isdigit((unsigned char)c)){
52         int start=lx->pos; bool isf=false;
53         while(isdigit((unsigned char)peek(lx))) adv(lx);
54         if(peek(lx)=='.'&&isdigit((unsigned char)peek2(lx))){ isf=true; adv(lx);
55             while(isdigit((unsigned char)peek(lx))) adv(lx); }
56         if(peek(lx)=='e' || peek(lx)=='E'){ isf=true; adv(lx);
57             if(peek(lx)=='+' || peek(lx)=='-')adv(lx);
58             while(isdigit((unsigned char)peek(lx))) adv(lx); }
59         int n=lx->pos-start; char*buf=malloc(n+1); memcpy(buf, lx->src+start,n); buf[n]=0;
60         Token t=mk(isf?T_FLOAT:T_INT,buf,line); t.num=atof(buf); free(buf); return t;
61     }
62     /* identifiers / keywords. allow trailing ? and ! and @ prefix */
63     if(isalpha((unsigned char)c)||c=='_'||c=='@'){
64         int start=lx->pos; if(c=='@')adv(lx);
65         while(isalnum((unsigned char)peek(lx))||peek(lx)=='_'||peek(lx)=='@') adv(lx);
66         if(peek(lx)=='?'||peek(lx)=='!') adv(lx);
67         int n=lx->pos-start; char*buf=malloc(n+1); memcpy(buf, lx->src+start,n); buf[n]=0;
68         if(buf[0]=='@'){
69             if(strcmp(buf,"@reviser")==0){ Token t=mk(T_REVISER,buf,line); free(buf); return t; }
70             Token t=mk(T_IDENT,buf,line); free(buf); return t;
71         }
72         if(strcmp(buf,"do")==0){ Token t=mk(T_IDENT,"do",line); free(buf); return t; }
73         for(int i=0;KEYWORDS[i].kw;i++) if(strcmp(buf,KEYWORDS[i].kw)==0 && KEYWORDS[i].t!=T_ERROR){
74             Token t=mk(KEYWORDS[i].t,buf,line); free(buf); return t; }
75         Token t=mk(T_IDENT,buf,line); free(buf); return t;
76     }
77     /* strings */
78     if(c==''){
79         adv(lx); int start=lx->pos;

```

```

80     while(peek(lx)&&peek(lx)!=''){ if(peek(lx)=='\\')adv(lx); adv(lx); }
81     int n=lx->pos-start; char*buf=malloc(n+1);
82     /* unescape minimally */
83     int j=0; for(int i=0;i<n;i++){ char ch=lx->src[start+i];
84         if(ch=='\\'){ i++; char e=lx->src[start+i];
85             buf[j++] = e=='n'?'\n': e=='t'?'\t': e=='"'?'\"': e=='\\'? '\\': e; }
86         else buf[j++] = ch; }
87     buf[j]=0; if(peek(lx)=='')adv(lx);
88     Token t=mk(T_STRING,buf,line); free(buf); return t;
89 }
90
91 adv(lx); /* consume c */
92 switch(c){
93     case '(' : return mk(T_LPAREN,"(",line);
94     case ')' : return mk(T_RPAREN,")",line);
95     case '{' : return mk(T_LBRACE,"{",line);
96     case '}' : return mk(T_RBRACE,"}",line);
97     case '[' : return mk(T_LBRACK,"[",line);
98     case ']' : return mk(T_RBRACK,"]",line);
99     case ',' : return mk(T_COMMA,",",line);
100    case '.' : return mk(T_DOT,".",line);
101    case '|' : return mk(T_PIPE,"|",line);
102    case '~' : return mk(T_CONSIST,"~",line);
103    case '+' : return mk(T_PLUS,"+",line);
104    case '*' : return mk(T_STAR_OP,"*",line);
105    case '/' : return mk(T_SLASH,"/",line);
106    case '%' : return mk(T_PERCENT,"%",line);
107    case ';' : return mk(T_SEMI,";",line);
108    case '-' :
109        if(peek(lx)=='>'){ adv(lx); return mk(T_QUERY,"->",line); } /* alt arrow */
110        return mk(T_MINUS,"-",line);
111    case '<' :
112        if(peek(lx)=='-'){ adv(lx); return mk(T_LARROW,"<-",line); }
113        if(peek(lx)=='='){ adv(lx); return mk(T_LE,"<=",line); }
114        return mk(T_LT,"<",line);
115    case '>' :
116        if(peek(lx)=='>'){ adv(lx); return mk(T_COMMIT,">>",line); }
117        if(peek(lx)=='='){ adv(lx); return mk(T_GE,">=",line); }
118        return mk(T_GT,">",line);
119    case '=' :
120        if(peek(lx)=='='){ adv(lx); return mk(T_EQ,"==",line); }
121        if(peek(lx)=='>'){ adv(lx); return mk(T_ARROW,">=",line); }
122        return mk(T_ASSIGN,"=",line);
123    case '!' :
124        if(peek(lx)=='='){ adv(lx); return mk(T_NE,"!=",line); }
125        return mk(T_NOT,"!",line);
126    case '&' :
127        if(peek(lx)=='&'){ adv(lx); return mk(T_AND,"&&",line); }
128        return mk(T_AND,"&",line);
129    case ':' :
130        if(peek(lx)=='='){ adv(lx);
131            if(peek(lx)=='>'){ adv(lx); return mk(T_DEFARROW,":=>",line); }
132            return mk(T_WALRUS,":=",line); }
133        if(peek(lx)==':') { adv(lx); return mk(T_SCOPE,"::",line); }
134        return mk(T_COLON,":",line);
135 }
136 if(c=='|'&&peek(lx)=='|'){ adv(lx); return mk(T_OR,"||",line); }
137 char b[2]={c,0};
138 return mk(T_ERROR,b,line);
139 }
140
141 const char* toktype_name(TokType t){
142     switch(t){
143         case T_INT: return "INT"; case T_FLOAT: return "FLOAT"; case T_STRING: return "STRING";
144         case T_IDENT: return "IDENT"; case T_COMMIT: return ">>"; case T_ARROW: return ">=";
145         case T_DEFARROW: return ":=>"; case T_WALRUS: return ":=>"; case T_LARROW: return "<-";
146         case T_CONSIST: return "~"; case T_SCOPE: return "::"; case T_REVISER: return "@reviser";
147         case T_EOF: return "EOF"; default: return "tok";
148     }
149 }

```

src/ast.c

AST node construction

```
1 #define _GNU_SOURCE
2 /* ast.c -- AST node construction. */
3 #include "bada.h"
4
5 Node *node_new(NodeType t,int line){
6     Node*n=calloc(1,sizeof(Node)); n->type=t; n->line=line; return n;
7 }
8 void node_push(Node*p,Node*c){
9     if(p->nkids>=p->cap){ p->cap=p->cap?p->cap*2:4;
10        p->kids=realloc(p->kids,p->cap*sizeof(Node*)); }
11     p->kids[p->nkids++]=c;
12 }
13 void node_free(Node*n){
14     if(!n)return;
15     for(int i=0;i<n->nkids;i++) node_free(n->kids[i]);
16     free(n->kids);
17     if(n->params){ for(int i=0;i<n->nparams;i++) free(n->params[i]); free(n->params); }
18     node_free(n->body);
19     free(n->str);
20     free(n);
21 }
```

src/parser.c

Recursive-descent parser

```

1 #define _GNU_SOURCE
2 /* parser.c -- recursive-descent parser for Bada.
3 *
4 * Grammar (informal, covering the PDF's Bada constructs):
5 *
6 * program      := decl*
7 * decl         := omegaDB | typedef | struct | func | stmt
8 * omegaDB      := 'Omega' '::' ('DATABASE'|'Tuplespace') ('[' IDENT ']')? block
9 * struct       := 'asperl' IDENT ':' 'struct' block
10 *              | 'asperl' IDENT block
11 * typedef      := 'typedef' (':' qualname)? block
12 * func         := 'def' IDENT pipeparams? ('>'|'=>'|':>'|'{'|...) body
13 * pipeparams:= '|' (IDENT (',' IDENT)*)? '|'
14 * lambda       := pipeparams expr (anonymous, used as call arg)
15 * stmt         := bind | assign | if | while | for | return | print | exprstmt
16 * bind         := IDENT ':' expr
17 * expr         := commit
18 * commit       := query ('>' query)*
19 * query        := assignment ('=>' assignment)* (also '->')
20 * ... standard precedence below ...
21 *
22 * The parser is permissive where the manuscript notation is loose; it records
23 * operator semantics (> commit, => query, <- append, ~ consistency) faithfully.
24 */
25 #include "bada.h"
26
27 typedef struct {
28     Token *toks; int n, i;
29     int errors;
30 } Parser;
31
32 static Token *cur(Parser*p){ return &p->toks[p->i]; }
33 static Token *peekn(Parser*p,int k){ int j=p->i+k; if(j>=p->n)j=p->n-1; return &p->toks[j]; }
34 static bool check(Parser*p,TokType t){ return cur(p)->type==t; }
35 static Token *advance(Parser*p){ Token*t=cur(p); if(p->i<p->n-1)p->i++; return t; }
36 static bool match(Parser*p,TokType t){ if(check(p,t)){advance(p);return true;} return false; }
37
38 static void perr(Parser*p,const char*msg){
39     fprintf(stderr,"[parse error] line %d near '%s': %s\n",
40         cur(p)->line, cur(p)->lexeme?cur(p)->lexeme:"", msg);
41     p->errors++;
42 }
43 static void expect(Parser*p,TokType t,const char*what){
44     if(!match(p,t)){ perr(p,what); }
45 }
46
47 /* forward */
48 static Node *parse_expr(Parser*p);
49 static Node *parse_stmt(Parser*p);
50 static Node *parse_block(Parser*p);
51 static Node *parse_decl(Parser*p);
52
53 static char *dupstr(const char*s){ return s?strdup(s):NULL; }
54
55 /* [a, b, c] -> param list. returns array of names via node->params */
56 static void parse_pipe_params(Parser*p, Node*holder){
57     expect(p,T_PIPE,"expected '|'");
58     while(!check(p,T_PIPE)&&!check(p,T_EOF)){
59         if(check(p,T_IDENT)){
60             holder->params=realloc(holder->params,(holder->nparams+1)*sizeof(char*));
61             holder->params[holder->nparams++]=dupstr(cur(p)->lexeme);
62             advance(p);
63         } else advance(p);
64         if(!match(p,T_COMMA)) {} /* tolerate */
65         if(check(p,T_PIPE)) break;
66         if(check(p,T_COMMA)) advance(p);
67     }
68     expect(p,T_PIPE,"expected closing '|'");
69 }
70
71 /* primary */
72 static Node *parse_primary(Parser*p){
73     Token*t=cur(p);
74     switch(t->type){
75         case T_INT:{ Node*n=node_new(N_INT,t->line); n->num=t->num; advance(p); return n; }
76         case T_FLOAT:{ Node*n=node_new(N_FLOAT,t->line); n->num=t->num; advance(p); return n; }
77         case T_STRING:{ Node*n=node_new(N_STRING,t->line); n->str=dupstr(t->lexeme); advance(p); return n; }
78         case T_TRUE:{ Node*n=node_new(N_BOOL,t->line); n->boolean=true; advance(p); return n; }
79         case T_FALSE:{ Node*n=node_new(N_BOOL,t->line); n->boolean=false; advance(p); return n; }

```

```

80     case T_NIL:{ Node*n=node_new(N_NIL,t->line); advance(p); return n; }
81     case T_STAR:{ Node*n=node_new(N_IDENT,t->line); n->str=dupstr("STAR"); advance(p); return n; }
82     case T_LPAREN:{ advance(p); Node*e=parse_expr(p); expect(p,T_RPAREN,"expected ')'"); return e; }
83     case T_LBRACK:{ /* array literal */
84         advance(p); Node*a=node_new(N_ARRAY,t->line);
85         while(!check(p,T_RBRACK)&&!check(p,T_EOF)){
86             node_push(a,parse_expr(p));
87             if(!match(p,T_COMMA)) break;
88         }
89         expect(p,T_RBRACK,"expected ']'");
90         return a;
91     }
92     case T_PIPE:{ /* lambda: |params| expr (or |params| do ... end) */
93         Node*lam=node_new(N_LAMBDA,t->line);
94         parse_pipe_params(p, lam);
95         if(check(p,T_IDENT)&&strcmp(cur(p)->lexeme,"do")==0){
96             advance(p);
97             Node*body=node_new(N_BLOCK,t->line);
98             while(!(check(p,T_IDENT)&&strcmp(cur(p)->lexeme,"end")==0)&&!check(p,T_EOF))
99                 node_push(body,parse_stmt(p));
100             if(check(p,T_IDENT)&&strcmp(cur(p)->lexeme,"end")==0) advance(p);
101             lam->body=body;
102         } else {
103             lam->body=parse_expr(p);
104         }
105         return lam;
106     }
107     case T_OMEGA:{ /* Omega used as a value/object reference */
108         Node*n=node_new(N_IDENT,t->line); n->str=dupstr("Omega"); advance(p);
109         return n;
110     }
111     case T_TUPLESPEACE:{ Node*n=node_new(N_IDENT,t->line); n->str=dupstr("TupleSpace"); advance(p); return n; }
112     case T_IDENT:{ Node*n=node_new(N_IDENT,t->line); n->str=dupstr(t->lexeme); advance(p); return n; }
113     case T_DATABASE:{ Node*n=node_new(N_IDENT,t->line); n->str=dupstr("DATABASE"); advance(p); return n; }
114     default:
115         perr(p,"expected expression");
116         advance(p);
117         return node_new(N_NIL,t->line);
118 }
119 }
120
121 /* postfix: calls f(...), index a[i], member a.b, scope A::B, a.each|x| ... */
122 static Node *parse_postfix(Parser*p){
123     Node*e=parse_primary(p);
124     for(;;){
125         if(match(p,T_LPAREN)){
126             Node*call=node_new(N_CALL,e->line); node_push(call,e);
127             while(!check(p,T_RPAREN)&&!check(p,T_EOF)){
128                 node_push(call,parse_expr(p));
129                 if(!match(p,T_COMMA)) break;
130             }
131             expect(p,T_RPAREN,"expected ')'"); e=call;
132         } else if(match(p,T_LBRACK)){
133             Node*idx=node_new(N_INDEX,e->line); node_push(idx,e);
134             if(!check(p,T_RBRACK)) node_push(idx,parse_expr(p));
135             expect(p,T_RBRACK,"expected ']'"); e=idx;
136         } else if(match(p,T_DOT)){
137             /* member, possibly .each |x| do..end / .each|x| expr or method call */
138             Token*m=cur(p);
139             if(check(p,T_EACH)){
140                 advance(p);
141                 Node*each=node_new(N_EACH,m->line); node_push(each,e); /* collection */
142                 Node*lam=node_new(N_LAMBDA,m->line);
143                 if(check(p,T_PIPE)) parse_pipe_params(p, lam);
144                 /* body: 'do ... end' or single expr */
145                 if(check(p,T_IDENT)&&strcmp(cur(p)->lexeme,"do")==0){
146                     advance(p);
147                     Node*body=node_new(N_BLOCK,m->line);
148                     while(!(check(p,T_IDENT)&&strcmp(cur(p)->lexeme,"end")==0)&&!check(p,T_EOF))
149                         node_push(body,parse_stmt(p));
150                     if(check(p,T_IDENT)&&strcmp(cur(p)->lexeme,"end")==0) advance(p);
151                     lam->body=body;
152                 } else {
153                     lam->body=parse_expr(p);
154                 }
155                 node_push(each, lam);
156                 e=each;
157             } else {
158                 char*name=dupstr(m->lexeme); advance(p);
159                 Node*get=node_new(N_GET,m->line); get->str=name; node_push(get,e);
160                 if(match(p,T_LPAREN)){
161                     Node*call=node_new(N_CALL,m->line);
162                     Node*meth=node_new(N_GET,m->line); meth->str=dupstr(name);
163                     node_push(meth,e);

```

```

164         /* replace get with a method-call form: CALL( GET(e,name), args ) */
165         node_free(get);
166         node_push(call, meth);
167         while(!check(p, T_RPAREN) && !check(p, T_EOF)){
168             node_push(call, parse_expr(p));
169             if(!match(p, T_COMMA)) break;
170         }
171         expect(p, T_RPAREN, "expected ')'");
172         e = call;
173     } else e = get;
174 }
175 } else if(match(p, T_SCOPE)){
176     /* A::B -> treat like member access on namespace */
177     Token*m = cur(p); char*name = dupstr(m->lexeme?m->lexeme:""); advance(p);
178     Node*get = node_new(N_GET, m->line); get->str = name; node_push(get, e);
179     e = get;
180 } else if(check(p, T_PIPE) && (e->type == N_IDENT || e->type == N_GET)){
181     /* call-with-pipe-args style: f |a, b| (manuscript: def f |x|) used in calls */
182     /* Only treat as call-args when clearly an argument list follows on same construct.
183        To stay safe, we DON'T auto-consume here; handled at def site. */
184     break;
185 } else break;
186 }
187 return e;
188 }
189
190 static Node *parse_unary(Parser*p){
191     if(check(p, T_MINUS) || check(p, T_NOT)){
192         Token*op = advance(p);
193         Node*n = node_new(N_UNARY, op->line); n->str = dupstr(op->lexeme);
194         node_push(n, parse_unary(p)); return n;
195     }
196     return parse_postfix(p);
197 }
198
199 static Node *bin(Parser*p, Node*(*sub)(Parser*), const TokType*ops, const char**names, int k){
200     Node*left = sub(p);
201     for(;;){
202         bool matched = false;
203         for(int i=0; i<k; i++) if(check(p, ops[i])){
204             Token*op = advance(p);
205             Node*n = node_new(N_BINARY, op->line); n->str = dupstr(names[i]);
206             node_push(n, left); node_push(n, sub(p)); left = n; matched = true; break;
207         }
208         if(!matched) break;
209     }
210     return left;
211 }
212
213 static Node *parse_factor(Parser*p){
214     static const TokType ops[] = {T_STAR_OP, T_SLASH, T_PERCENT};
215     static const char*nm[] = {"*", "/", "%"};
216     return bin(p, parse_unary, ops, nm, 3);
217 }
218
219 static Node *parse_term(Parser*p){
220     static const TokType ops[] = {T_PLUS, T_MINUS};
221     static const char*nm[] = {"+", "-"};
222     return bin(p, parse_factor, ops, nm, 2);
223 }
224
225 static Node *parse_compare(Parser*p){
226     static const TokType ops[] = {T_LT, T_LE, T_GT, T_GE};
227     static const char*nm[] = {"<", "<=", ">", ">="};
228     return bin(p, parse_term, ops, nm, 4);
229 }
230
231 static Node *parse_equality(Parser*p){
232     static const TokType ops[] = {T_EQ, T_NE, T_CONSIST};
233     static const char*nm[] = {"==", "!=", "~"};
234     return bin(p, parse_compare, ops, nm, 3);
235 }
236
237 static Node *parse_and(Parser*p){
238     static const TokType ops[] = {T_AND}; static const char*nm[] = {"&&"};
239     return bin(p, parse_equality, ops, nm, 1);
240 }
241
242 static Node *parse_or(Parser*p){
243     static const TokType ops[] = {T_OR}; static const char*nm[] = {"||"};
244     return bin(p, parse_and, ops, nm, 1);
245 }
246
247 /* query level: a => b and a -> b (pattern query / map result) */
248 static Node *parse_query(Parser*p){
249     Node*left = parse_or(p);
250     while(check(p, T_ARROW) || check(p, T_QUERY)){
251         Token*op = advance(p);
252         Node*n = node_new(N_QUERY, op->line);

```

```

248     node_push(n, left); node_push(n, parse_or(p)); left=n;
249 }
250 return left;
251 }
252
253 /* commit level: a >> b (commit / append to ledger), and facts <- fact */
254 static Node *parse_commit(Parser*p){
255     Node*left=parse_query(p);
256     for(;;){
257         if(check(p, T_COMMIT)){
258             advance(p);
259             Node*n=node_new(N_COMMIT, left->line);
260             node_push(n, left); node_push(n, parse_query(p)); left=n;
261         } else if(check(p, T_LARROW)){
262             advance(p);
263             Node*n=node_new(N_LARROW, left->line);
264             node_push(n, left); node_push(n, parse_query(p)); left=n;
265         } else break;
266     }
267     return left;
268 }
269
270 static Node *parse_expr(Parser*p){
271     /* assignment / bind handled at statement level; expr starts at commit */
272     return parse_commit(p);
273 }
274
275 /* ---- statements ---- */
276 static Node *parse_block(Parser*p){
277     int line=cur(p)->line;
278     expect(p, T_LBRACE, "expected '{'");
279     Node*b=node_new(N_BLOCK, line);
280     while(!check(p, T_RBRACE)&&!check(p, T_EOF))
281         node_push(b, parse_decl(p));
282     expect(p, T_RBRACE, "expected '}'");
283     return b;
284 }
285
286 static Node *parse_func(Parser*p){
287     int line=cur(p)->line;
288     expect(p, T_DEF, "expected 'def'");
289     Node*fn=node_new(N_FUNC, line);
290     if(check(p, T_IDENT)){ fn->str=dupstr(cur(p)->lexeme); advance(p); }
291     else if(check(p, T_GET)){ fn->str=dupstr("anon"); }
292     else { fn->str=dupstr("anon"); }
293     /* params: either |a,b| or (a,b) */
294     if(check(p, T_PIPE)) parse_pipe_params(p, fn);
295     else if(match(p, T_LPAREN)){
296         while(!check(p, T_RPAREN)&&!check(p, T_EOF)){
297             if(check(p, T_IDENT)){
298                 fn->params=realloc(fn->params, (fn->nparams+1)*sizeof(char*));
299                 fn->params[fn->nparams++]=dupstr(cur(p)->lexeme); advance(p);
300             } else advance(p);
301             if(!match(p, T_COMMA)) break;
302         }
303         expect(p, T_RPAREN, "expected ')'");
304     }
305     /* body connector: >> expr | => expr | { block } | :=> expr */
306     if(check(p, T_LBRACE)){
307         fn->body=parse_block(p);
308     } else if(match(p, T_COMMIT)||match(p, T_ARROW)||match(p, T_DEFARROW)||match(p, T_ASSIGN)){
309         Node*body=node_new(N_BLOCK, line);
310         Node*ret=node_new(N_RETURN, line);
311         node_push(ret, parse_expr(p));
312         node_push(body, ret);
313         match(p, T_SEMI);
314         fn->body=body;
315     } else {
316         /* allow bare expression body */
317         Node*body=node_new(N_BLOCK, line);
318         Node*ret=node_new(N_RETURN, line);
319         node_push(ret, parse_expr(p));
320         node_push(body, ret);
321         fn->body=body;
322     }
323     return fn;
324 }
325
326 /* asperal Name :struct { ... } or asperal Name { ... }
327 also: asperal |term| :=> { ... } (the gradual-typing union form) */
328 static Node *parse_asperal(Parser*p){
329     int line=cur(p)->line;
330     expect(p, T_ASPERAL, "expected 'asperal'");
331     if(check(p, T_PIPE)){

```

```

332     /* asperal |term| :=> { union of kinds } : we capture as a struct-ish node */
333     Node*n=node_new(N_ASUPERAL, line); n->str=dupstr("union");
334     parse_pipe_params(p,n);
335     match(p,T_DEFARROW);
336     if(check(p,T_LBRACE)) n->body=parse_block(p);
337     return n;
338 }
339 Node*st=node_new(N_STRUCT, line);
340 if(check(p,T_IDENT)){ st->str=dupstr(cur(p)->lexeme); advance(p); }
341 else if(check(p,T_TUPLESPEACE)){ st->str=dupstr("TupleSpace"); advance(p); }
342 else st->str=dupstr("anon");
343 /* optional ':' 'struct' */
344 if(match(p,T_COLON)) match(p,T_STRUCT);
345 else match(p,T_STRUCT);
346 st->body=parse_block(p);
347 return st;
348 }
349
350 /* Omega :: (DATABASE|Tuplespace) [tag]? { ... } -- engine namespace block */
351 static Node *parse_omega(Parser*p){
352     int line=cur(p)->line;
353     expect(p,T_OMEGA,"expected 'Omega'");
354     Node*n=node_new(N_OMEGA_DB, line);
355     if(match(p,T_SCOPE)){
356         if(check(p,T_DATABASE)){ advance(p); n->str=dupstr("DATABASE"); }
357         else if(check(p,T_TUPLESPEACE)){ advance(p); n->str=dupstr("Tuplespace"); }
358         else if(check(p,T_IDENT)){ n->str=dupstr(cur(p)->lexeme); advance(p); }
359         else n->str=dupstr("DATABASE");
360     } else n->str=dupstr("DATABASE");
361     if(match(p,T_LBRACK)){ if(check(p,T_TUPLESPEACE)||check(p,T_IDENT)) advance(p);
362         expect(p,T_RBRACK,"expected ']'"); }
363     /* optional '<' DATABASE (Omega::Tuplespace < DATABASE) */
364     if(match(p,T_LT)){ if(check(p,T_DATABASE)||check(p,T_IDENT)) advance(p); }
365     n->body=parse_block(p);
366     return n;
367 }
368
369 /* @reviser : def < Omega::...{ ... } -- reflective learning loop block */
370 static Node *parse_reviser(Parser*p){
371     int line=cur(p)->line;
372     expect(p,T_REVISER,"expected '@reviser'");
373     Node*n=node_new(N_REVISER, line);
374     /* skip ': def < Omega::DATABASE[tuplespace].mechanism' header noise up to '{' */
375     while(!check(p,T_LBRACE)&&!check(p,T_EOF)) advance(p);
376     n->body=parse_block(p);
377     return n;
378 }
379
380 static Node *parse_if(Parser*p){
381     int line=cur(p)->line; expect(p,T_IF,"if");
382     Node*n=node_new(N_IF, line);
383     bool par=match(p,T_LPAREN);
384     node_push(n,parse_expr(p));
385     if(par) expect(p,T_RPAREN,"");
386     node_push(n,parse_block(p));
387     if(match(p,T_ELSE)){
388         if(check(p,T_IF)) node_push(n,parse_if(p));
389         else node_push(n,parse_block(p));
390     }
391     return n;
392 }
393
394 static Node *parse_while(Parser*p){
395     int line=cur(p)->line; expect(p,T_WHILE,"while");
396     Node*n=node_new(N_WHILE, line);
397     bool par=match(p,T_LPAREN);
398     node_push(n,parse_expr(p));
399     if(par) expect(p,T_RPAREN,"");
400     node_push(n,parse_block(p));
401     return n;
402 }
403
404 static Node *parse_for(Parser*p){
405     int line=cur(p)->line; expect(p,T_FOR,"for");
406     Node*n=node_new(N_FOR, line);
407     n->str=dupstr(cur(p)->lexeme); /* loop var */
408     if(check(p,T_IDENT)) advance(p);
409     expect(p,T_IN,"in");
410     node_push(n,parse_expr(p)); /* iterable */
411     node_push(n,parse_block(p));
412     return n;
413 }
414
415 static Node *parse_stmt(Parser*p){
416     int line=cur(p)->line;
417     if(check(p,T_DEF)) return parse_func(p);

```

```

416     if(check(p,T_ASERAL)) return parse_asperal(p);
417     if(check(p,T_OMEGA)) return parse_omega(p);
418     if(check(p,T_REVISER)) return parse_reviser(p);
419     if(check(p,T_IF)) return parse_if(p);
420     if(check(p,T_WHILE)) return parse_while(p);
421     if(check(p,T_FOR)) return parse_for(p);
422     if(check(p,T_TYPEDEF)){ advance(p);
423         /* typedef [:qual] { ... } -- record the body as a struct-like decl */
424         Node*n=node_new(N_STRUCT,line); n->str=dupstr("typedef");
425         while(!check(p,T_LBRACE)&&!check(p,T_EOF)) advance(p);
426         n->body=parse_block(p); return n; }
427     if(match(p,T_RETURN)){
428         Node*n=node_new(N_RETURN,line);
429         if(!check(p,T_RBRACE)&&!check(p,T_SEMI)) node_push(n,parse_expr(p));
430         match(p,T_SEMI); return n;
431     }
432     if(match(p,T_PRINT)){
433         Node*n=node_new(N_PRINT,line);
434         bool par=match(p,T_LPAREN);
435         node_push(n,parse_expr(p));
436         while(match(p,T_COMMA)) node_push(n,parse_expr(p));
437         if(par) expect(p,T_RPAREN,"");
438         match(p,T_SEMI); return n;
439     }
440     /* bind: IDENT := expr / assign: lvalue = expr / exprstmt */
441     if(check(p,T_IDENT)&&peekn(p,1)->type==T_WALRUS){
442         Node*n=node_new(N_BIND,line); n->str=dupstr(cur(p)->lexeme);
443         advance(p); advance(p); /* IDENT := */
444         node_push(n,parse_expr(p));
445         match(p,T_SEMI); return n;
446     }
447     Node*e=parse_expr(p);
448     if(check(p,T_ASSIGN)){
449         advance(p);
450         Node*n=node_new(N_ASSIGN,line);
451         node_push(n,e); node_push(n,parse_expr(p));
452         match(p,T_SEMI); return n;
453     }
454     match(p,T_SEMI);
455     return e;
456 }
457
458 static Node *parse_decl(Parser*p){
459     return parse_stmt(p);
460 }
461
462 Node *parse_program(const char*src){
463     /* tokenize fully */
464     Lexer lx; lexer_init(&lx,src);
465     int cap=256,n=0; Token*toks=malloc(cap*sizeof(Token));
466     for(;;){
467         if(n==cap){ cap*=2; toks=realloc(toks,cap*sizeof(Token)); }
468         toks[n]=lexer_next(&lx);
469         if(toks[n].type==T_EOF){ n++; break; }
470         n++;
471     }
472     Parser p={toks,n,0,0};
473     Node*prog=node_new(N_PROGRAM,1);
474     while(!check(&p,T_EOF)){
475         Node*d=parse_decl(&p);
476         if(d) node_push(prog,d);
477         if(p.errors>50){ fprintf(stderr,"too many parse errors, aborting\n"); break; }
478     }
479     for(int i=0;i<n;i++) token_free(&toks[i]);
480     free(toks);
481     if(p.errors){ fprintf(stderr,"parser finished with %d error(s)\n",p.errors); }
482     return prog;
483 }

```

src/interp.c

Tree-walking interpreter + engine builtins

```

1 #define _GNU_SOURCE
2 /* interp.c -- tree-walking interpreter for Bada.
3  *
4  * Native value model:
5  *   - numbers (double), bools, strings, nil
6  *   - arrays
7  *   - dist : probability vector (native; carries V doubles, normalised)
8  *   - phase : complex unit-modulus vector (native; V complex doubles)
9  *   - tuplespace : the append-only immutable ledger object
10  *   - functions / lambdas (closures over the defining environment)
11  *   - structs (object templates) and instances (field maps)
12  *
13  * The Unknown-Prior Engine builtins (softmax, entropy, unknown_prior, update,
14  * manifold_embed, cognitive_system) are wired in as native functions so the
15  * Bada source from the manuscript runs with the documented numerics, including
16  * the two provable properties (zero preservation;  $|\psi|^2 == \text{base } a$ ).
17  */
18 #include "bada.h"
19
20 /* ----- values ----- */
21 typedef enum { V_NIL, V_NUM, V_BOOL, V_STR, V_ARR, V_DIST, V_PHASE,
22               V_FUNC, V_NATIVE, V_TUPLESPEACE, V_STRUCT, V_INST, V_STAR } VType;
23
24 typedef struct Value Value;
25 typedef struct Env Env;
26
27 typedef Value (*NativeFn)(Value*args, int n, Env*env);
28
29 typedef struct { Value*items; int len, cap; } Arr;
30
31 typedef struct {          /* append-only ledger */
32     Value *facts;         /* immutable snapshots */
33     int len, cap;
34 } TupleSpace;
35
36 typedef struct {          /* dist: native probability vector */
37     double *p; int v;
38 } Dist;
39
40 typedef struct {          /* phase: native complex unit-modulus vector */
41     double complex *z; int v;
42 } Phase;
43
44 struct Value {
45     VType t;
46     union {
47         double num;
48         bool b;
49         char *str;
50         Arr *arr;
51         Dist *dist;
52         Phase *phase;
53         TupleSpace *ts;
54         struct { Node*decl; Env*closure; } fn; /* user function/lambda */
55         NativeFn native;
56         struct { char*name; Node*decl; Env*closure; } st; /* struct template */
57         struct { char*name; Env*fields; } inst; /* instance */
58     } as;
59 };
60
61 static Value V_nil(){ Value v; v.t=V_NIL; return v; }
62 static Value V_num(double x){ Value v; v.t=V_NUM; v.as.num=x; return v; }
63 static Value V_bool(bool b){ Value v; v.t=V_BOOL; v.as.b=b; return v; }
64 static Value V_str(const char*s){ Value v; v.t=V_STR; v.as.str=strdup(s?s:""); return v; }
65 static Value V_star(){ Value v; v.t=V_STAR; return v; }
66
67 /* ----- env ----- */
68 typedef struct { char*key; Value val; } Entry;
69 struct Env {
70     Entry *e; int len, cap;
71     Env *parent;
72 };
73 static Env *env_new(Env*parent){ Env*e=malloc(1, sizeof(Env)); e->parent=parent; return e; }
74 static void env_set(Env*env, const char*k, Value v){
75     for(int i=0; i<env->len; i++) if(strcmp(env->e[i].key, k)==0){ env->e[i].val=v; return; }
76     if(env->len==env->cap){ env->cap=env->cap?env->cap*2:8; env->e=realloc(env->e, env->cap*sizeof(Entry)); }
77     env->e[env->len].key=strdup(k); env->e[env->len].val=v; env->len++;
78 }
79 static bool env_get(Env*env, const char*k, Value*out){

```

```

80     for(Env*e=env;e;e=e->parent)
81         for(int i=0;i<e->len;i++) if(strcmp(e->e[i].key,k)==0){ *out=e->e[i].val; return true; }
82     return false;
83 }
84
85 /* ----- dist / phase helpers ----- */
86 static Dist *dist_alloc(int v){ Dist*d=malloc(sizeof(Dist)); d->p=calloc(v,sizeof(double)); d->v=v; return d; }
87 static Value V_dist_take(Dist*d){ Value v; v.t=V_DIST; v.as.dist=d; return v; }
88 static Phase*phase_alloc(int v){ Phase*p=malloc(sizeof(Phase)); p->z=calloc(v,sizeof(double complex)); p->v=v; return p; }
89 static Value V_phase_take(Phase*p){ Value v; v.t=V_PHASE; v.as.phase=p; return v; }
90
91 static void dist_normalize(Dist*d){
92     double s=0; for(int i=0;i<d->v;i++) s+=d->p[i];
93     if(s>0) for(int i=0;i<d->v;i++) d->p[i]/=s;
94 }
95
96 /* ----- arrays ----- */
97 static Arr *arr_new(){ Arr*a=calloc(1,sizeof(Arr)); return a; }
98 static void arr_push(Arr*a,Value v){
99     if(a->len>=a->cap){ a->cap=a->cap?a->cap*2:8; a->items=realloc(a->items,a->cap*sizeof(Value)); }
100    a->items[a->len++]=v;
101 }
102 static Value V_arr_take(Arr*a){ Value v; v.t=V_ARR; v.as.arr=a; return v; }
103
104 /* convert array<num> to dist (max-entropy / raw vector) */
105 static Dist *arr_to_dist(Arr*a){
106     Dist*d=dist_alloc(a->len);
107     for(int i=0;i<a->len;i++) d->p[i]= a->items[i].t==V_NUM? a->items[i].as.num:0;
108     return d;
109 }
110
111 /* ----- truthiness / print ----- */
112 static bool truthy(Value v){
113     switch(v.t){ case V_NIL:case V_STAR:return false; case V_BOOL:return v.as.b;
114                 case V_NUM:return v.as.num!=0; case V_STR:return v.as.str&&v.as.str[0];
115                 default:return true; }
116 }
117
118 static void print_value(Value v){
119     switch(v.t){
120         case V_NIL: printf("nil"); break;
121         case V_STAR: printf("star"); break;
122         case V_BOOL: printf(v.as.b?"true":"false"); break;
123         case V_NUM:
124             if(v.as.num==(long long)v.as.num) printf("%lld",(long long)v.as.num);
125             else printf("%.5f",v.as.num);
126             break;
127         case V_STR: printf("%s",v.as.str); break;
128         case V_ARR:{ printf("[");
129                     for(int i=0;i<v.as.arr->len;i++){ if(i)printf(", "); print_value(v.as.arr->items[i]); }
130                     printf("]"); break; }
131         case V_DIST:{ printf("dist[");
132                     for(int i=0;i<v.as.dist->v;i++){ if(i)printf(", "); printf("%.5f",v.as.dist->p[i]); }
133                     printf("]"); break; }
134         case V_PHASE:{ printf("phase[V=%d, |.|=1]",v.as.phase->v); break; }
135         case V_TUPLESPEACE: printf("tuplespace<%d facts>",v.as.ts?v.as.ts->len:0); break;
136         case V_FUNC: printf("<fn %s>",v.as.fn.decl&&v.as.fn.decl->str?v.as.fn.decl->str:"lambda"); break;
137         case V_NATIVE: printf("<native fn>"); break;
138         case V_STRUCT: printf("<struct %s>",v.as.st.name); break;
139         case V_INST: printf("<%s instance>",v.as.inst.name); break;
140     }
141 }
142
143 /* ----- control-flow signal ----- */
144 typedef struct { bool ret; Value val; } Flow;
145 static Flow flow_none(){ Flow f; f.ret=false; f.val=V_nil(); return f; }
146
147 /* forward */
148 static Value eval(Node*n,Env*env);
149 static Flow exec(Node*n,Env*env);
150 static Value call_value(Value callee,Value*args,int n,Env*env);
151
152 /* the engine's single global ledger (also addressable as Omega) */
153 static TupleSpace *G_LEDGER=NULL;
154
155 /* ===== NATIVE BUILTINS ===== */
156 /* numeric coercion helper */
157 static double as_num(Value v){ return v.t==V_NUM? v.as.num : 0; }
158
159 /* softmax(arr|dist) -> dist */
160 static Value nf_softmax(Value*a,int n,Env*env){ (void)env;
161     Dist*src=NULL; int v;
162     if(n<1) return V_nil();

```

```

163     if(a[0].t==V_DIST){ v=a[0].as.dist->v; src=a[0].as.dist; }
164     else if(a[0].t==V_ARR){ Dist*d=arr_to_dist(a[0].as.arr); src=d; v=d->v; }
165     else return V_nil();
166     Dist*out=dist_alloc(v);
167     double mx=-INFINITY; for(int i=0;i<v;i++) if(src->p[i]>mx)mx=src->p[i];
168     double s=0; for(int i=0;i<v;i++){ out->p[i]=exp(src->p[i]-mx); s+=out->p[i]; }
169     for(int i=0;i<v;i++) out->p[i]/=s;
170     /* confident-zero semantics: a logit far below the max is a ruled-out token
171        (probability exactly 0), matching the manuscript's a_i = 0. */
172     for(int i=0;i<v;i++) if(src->p[i]-mx < -30.0) out->p[i]=0.0;
173     dist_normalize(out);
174     if(a[0].t==V_ARR){ free(src->p); free(src); }
175     return V_dist_take(out);
176 }
177
178 /* entropy(dist|arr) -> num (Shannon, natural log) */
179 static Value nf_entropy(Value*a,int n,Env*env){ (void)env; (void)n;
180     double *p; int v; bool owned=false;
181     if(a[0].t==V_DIST){ p=a[0].as.dist->p; v=a[0].as.dist->v; }
182     else if(a[0].t==V_ARR){ Dist*d=arr_to_dist(a[0].as.arr); p=d->p; v=d->v; owned=true;
183         free(d); /* keep p */ p=NULL; Dist*dd=arr_to_dist(a[0].as.arr); p=dd->p; v=dd->v; }
184     else return V_num(0);
185     double H=0; for(int i=0;i<v;i++){ double q=p[i]; if(q<1e-12)q=1e-12; H+= -q*log(q); }
186     if(owned) free(p);
187     return V_num(H);
188 }
189
190 /* unknown_prior(V) -> dist (uniform = max entropy, Jaynes) */
191 static Value nf_unknown_prior(Value*a,int n,Env*env){ (void)env; (void)n;
192     int v=(int)as_num(a[0]); if(v<1)v=1;
193     Dist*d=dist_alloc(v); for(int i=0;i<v;i++) d->p[i]=1.0/v;
194     return V_dist_take(d);
195 }
196
197 /* update(prior, evidence [,lr]) -> dist (sharpen) */
198 static Value nf_update(Value*a,int n,Env*env){ (void)env;
199     if(n<2||a[0].t!=V_DIST) return n>0?a[0]:V_nil();
200     Dist*prior=a[0].as.dist;
201     Dist*evld = a[1].t==V_DIST? a[1].as.dist : (a[1].t==V_ARR? arr_to_dist(a[1].as.arr):NULL);
202     if(!evld) return a[0];
203     double lr = n>=3? as_num(a[2]) : 0.5;
204     int v=prior->v; Dist*out=dist_alloc(v);
205     double s=0;
206     for(int i=0;i<v;i++){ double e= i<evld->v? evld->p[i]:0; out->p[i]=(1-lr)*prior->p[i]+lr*e; s+=out->p[i]; }
207     if(s>0) for(int i=0;i<v;i++) out->p[i]/=s;
208     if(a[1].t==V_ARR){ free(evld->p); free(evld); }
209     return V_dist_take(out);
210 }
211
212 /* manifold_embed(arr x, arr-of-arr W) -> dist (L2-normalised linear embed) */
213 static Value nf_manifold_embed(Value*a,int n,Env*env){ (void)env;
214     if(n<1||a[0].t!=V_ARR) return V_nil();
215     Arr*x=a[0].as.arr;
216     Arr*W = (n>=2&&a[1].t==V_ARR)? a[1].as.arr : NULL;
217     int outdim = W? W->len : x->len;
218     Dist*d=dist_alloc(outdim);
219     for(int i=0;i<outdim;i++){
220         double acc=0;
221         if(W && W->items[i].t==V_ARR){
222             Arr*row=W->items[i].as.arr;
223             for(int j=0;j<x->len && j<row->len;j++) acc+= as_num(row->items[j])*as_num(x->items[j]);
224             } else { acc = as_num(x->items[i]); }
225         d->p[i]=acc;
226     }
227     double nrm=0; for(int i=0;i<outdim;i++) nrm+=d->p[i]*d->p[i];
228     nrm=sqrt(nrm)+1e-12; for(int i=0;i<outdim;i++) d->p[i]/=nrm;
229     return V_dist_take(d);
230 }
231
232 /* cognitive_system(base, modules, betas) -> dist q_hat
233    * base : arr (logits) or dist (already softmaxed -> treated as a directly)
234    * modules: arr of [q_m(dist|arr), phi_m(arr)] pairs
235    * betas : arr of nums
236    * Implements psi = sqrt(a)*exp(i*theta); q_hat = |psi|^2 / sum, == a exactly.
237    * Commits q_hat to the global ledger.
238    */
239 static Value nf_cognitive_system(Value*a,int n,Env*env){ (void)env;
240     if(n<1) return V_nil();
241     /* obtain a = softmax(base) (or base if already a dist) */
242     Dist*adist;
243     if(a[0].t==V_DIST){ int v=a[0].as.dist->v; adist=dist_alloc(v);
244         for(int i=0;i<v;i++) adist->p[i]=a[0].as.dist->p[i]; }
245     else { Value s=nf_softmax(&a[0],1,env); adist=dist_alloc(s.as.dist->v);
246         for(int i=0;i<adist->v;i++) adist->p[i]=s.as.dist->p[i];

```

```

247     free(s.as.dist->p); free(s.as.dist); }
248     int v=adist->v;
249     double *theta=calloc(v,sizeof(double));
250
251     Arr*mods = (n>=2&&a[1].t==V_ARR)? a[1].as.arr : NULL;
252     Arr*betas= (n>=3&&a[2].t==V_ARR)? a[2].as.arr : NULL;
253     if(mods){
254         for(int m=0;m<mods->len;m++){
255             if(mods->items[m].t!=V_ARR) continue;
256             Arr*pair=mods->items[m].as.arr;
257             if(pair->len<2) continue;
258             /* q_m */
259             double *q; int qv; bool ownq=false;
260             if(pair->items[0].t==V_DIST){ q=pair->items[0].as.dist->p; qv=pair->items[0].as.dist->v; }
261             else if(pair->items[0].t==V_ARR){ Dist*dd=arr_to_dist(pair->items[0].as.arr); q=dd->p; qv=dd->v; ownq=true; free(dd); }
262             else continue;
263             /* H_m = Shannon entropy */
264             double H=0; for(int i=0;i<qv;i++){ double qq=q[i]; if(qq<1e-12)qq=1e-12; H+= -qq*log(qq); }
265             /* phi_m */
266             Arr*phi = pair->items[1].t==V_ARR? pair->items[1].as.arr : NULL;
267             double beta = (betas && m<betas->len)? as_num(betas->items[m]) : 1.0;
268             for(int i=0;i<qv;i++){
269                 double phival = (phi && i<phi->len)? as_num(phi->items[i]) : 0;
270                 theta[i]+= beta*H*phival;
271             }
272             if(ownq){ /* q points into freed dist's array? we freed only struct, keep p */ free(q); }
273         }
274     }
275     /* psi = sqrt(a)*exp(i theta); |psi|^2 = a exactly */
276     Dist*qhat=dist_alloc(v);
277     double s=0;
278     for(int i=0;i<v;i++){
279         double complex psi = sqrt(adist->p[i]) * cexp(I*theta[i]);
280         double mag2 = creal(psi)*creal(psi)+cimag(psi)*cimag(psi);
281         qhat->p[i]=mag2; s+=mag2;
282     }
283     if(s>0) for(int i=0;i<v;i++) qhat->p[i]/=s;
284     free(theta);
285
286     /* commit q_hat to ledger (append-only) */
287     if(G_LEDGER){
288         Value snap; snap.t=V_DIST; snap.as.dist=dist_alloc(v);
289         for(int i=0;i<v;i++) snap.as.dist->p[i]=qhat->p[i];
290         if(G_LEDGER->len>=G_LEDGER->cap){ G_LEDGER->cap=G_LEDGER->cap*2*8;
291             G_LEDGER->facts=realloc(G_LEDGER->facts,G_LEDGER->cap*sizeof(Value)); }
292         G_LEDGER->facts[G_LEDGER->len++]=snap;
293     }
294     /* stash base 'a' for verification under env global __last_a */
295     Value av; av.t=V_DIST; av.as.dist=adist;
296     /* store in global root env via a side channel: keep as a field on result?
297     We return qhat and also set __last_a in the topmost env. */
298     Env*root=env; while(root->parent) root=root->parent;
299     env_set(root,"__last_a",av);
300     return V_dist_take(qhat);
301 }
302
303 /* len(x) */
304 static Value nf_len(Value*a,int n,Env*env){ (void)env;(void)n;
305     switch(a[0].t){ case V_ARR:return V_num(a[0].as.arr->len);
306         case V_DIST:return V_num(a[0].as.dist->v);
307         case V_STR:return V_num(strlen(a[0].as.str));
308         case V_TUPLESPACE:return V_num(a[0].as.ts?a[0].as.ts->len:0);
309         default:return V_num(0); }
310 }
311 /* sqrt, log, exp, abs */
312 static Value nf_sqrt(Value*a,int n,Env*e){(void)e;(void)n;return V_num(sqrt(as_num(a[0])));}
313 static Value nf_log (Value*a,int n,Env*e){(void)e;(void)n;return V_num(log(as_num(a[0])));}
314 static Value nf_exp (Value*a,int n,Env*e){(void)e;(void)n;return V_num(exp(as_num(a[0])));}
315 static Value nf_abs (Value*a,int n,Env*e){(void)e;(void)n;double x=as_num(a[0]);return V_num(x<0?-x:x);}
316
317 /* dist(arr) -> dist (construct native dist, normalised) */
318 static Value nf_dist(Value*a,int n,Env*e){(void)e;(void)n;
319     if(a[0].t==V_ARR){ Dist*d=arr_to_dist(a[0].as.arr); dist_normalize(d); return V_dist_take(d); }
320     if(a[0].t==V_DIST) return a[0];
321     return V_nil();
322 }
323 /* zeros_of(dist) -> arr of indices where p==0 */
324 static Value nf_zeros_of(Value*a,int n,Env*e){(void)e;(void)n;
325     if(a[0].t!=V_DIST) return V_arr_take(arr_new());
326     Arr*out=arr_new();
327     for(int i=0;i<a[0].as.dist->v;i++) if(a[0].as.dist->p[i]==0.0) arr_push(out,V_num(i));
328     return V_arr_take(out);
329 }

```

```

330 /* maxdiff(dist a, dist b) -> num (max |a_i - b_i|) for property (ii) */
331 static Value nf_maxdiff(Value*a,int n,Env*e){(void)e;(void)n;
332   if(a[0].t!=V_DIST||a[1].t!=V_DIST) return V_num(0);
333   int v=a[0].as.dist->v; if(a[1].as.dist->v<v) v=a[1].as.dist->v;
334   double m=0; for(int i=0;i<v;i++){ double d=a[0].as.dist->p[i]-a[1].as.dist->p[i];
335     if(d<0)d=-d; if(d>m)m=d; }
336   return V_num(m);
337 }
338 /* sci(num) -> string in scientific notation (for the 5.6e-17 line) */
339 static Value nf_sci(Value*a,int n,Env*e){(void)e;(void)n;
340   char buf[64]; sprintf(buf,64,"%e",as_num(a[0])); return V_str(buf);
341 }
342 /* f5(num) -> string fixed to 5 decimals (matches the manuscript table) */
343 static Value nf_f5(Value*a,int n,Env*e){(void)e;(void)n;
344   char buf[64]; sprintf(buf,64,"%5f",as_num(a[0])); return V_str(buf);
345 }
346 /* ledger() -> the global tuplespace */
347 static Value nf_ledger(Value*a,int n,Env*e){(void)a;(void)n;(void)e;
348   Value v; v.t=V_TUPLESPEC; v.as.ts=G_LEDGER; return v; }
349
350 /* last_a() -> dist : the base distribution a used in the last cognitive_system
351   call (before phase), for verifying property (ii) |psi|^2 == a. */
352 static Value nf_last_a(Value*a,int n,Env*e){(void)a;(void)n;
353   Env*root=e; while(root->parent) root=root->parent;
354   Value v; if(env_get(root,"__last_a",&v)) return v; return V_nil();
355 }
356 static void install_natives(Env*g){
357   struct { const char*name; NativeFn fn; } tab[]={
358     {"softmax",nf_softmax},{ "entropy",nf_entropy},{ "unknown_prior",nf_unknown_prior},
359     {"update",nf_update},{ "manifold_embed",nf_manifold_embed},
360     {"cognitive_system",nf_cognitive_system},{ "len",nf_len},
361     {"sqrt",nf_sqrt},{ "log",nf_log},{ "exp",nf_exp},{ "abs",nf_abs},
362     {"dist",nf_dist},{ "zeros_of",nf_zeros_of},{ "ledger",nf_ledger},
363     {"maxdiff",nf_maxdiff},{ "sci",nf_sci},{ "last_a",nf_last_a},{ "f5",nf_f5},
364     {NULL,NULL}
365   };
366   for(int i=0;tab[i].name;i++){ Value v; v.t=V_NATIVE; v.as.native=tab[i].fn; env_set(g,tab[i].name,v); }
367 }
368
369 /* ===== EVALUATION ===== */
370 static Value eval_binary(const char*op,Value l,Value r){
371   if(strcmp(op,"~")==0){ /* gradual consistency: STAR ~ anything, else equality */
372     if(l.t==V_STAR||r.t==V_STAR) return V_bool(true);
373     if(l.t==V_NUM&&r.t==V_NUM) return V_bool(l.as.num==r.as.num);
374     if(l.t==V_STR&&r.t==V_STR) return V_bool(strcmp(l.as.str,r.as.str)==0);
375     return V_bool(l.t==r.t);
376   }
377   if(strcmp(op,"&&")==0) return V_bool(truthy(l)&&truthy(r));
378   if(strcmp(op,"||")==0) return V_bool(truthy(l)||truthy(r));
379   if(strcmp(op,"")==0){
380     if(l.t==V_NUM&&r.t==V_NUM) return V_bool(l.as.num==r.as.num);
381     if(l.t==V_STR&&r.t==V_STR) return V_bool(strcmp(l.as.str,r.as.str)==0);
382     return V_bool(l.t==r.t);
383   }
384   if(strcmp(op,"!=")==0){ Value e=eval_binary("=",l,r); return V_bool(!e.as.b); }
385   /* string concat */
386   if(strcmp(op,"+")==0 && (l.t==V_STR||r.t==V_STR)){
387     char buf[512]; char a[256],b[256];
388     if(l.t==V_STR) sprintf(a,256,"%s",l.as.str); else sprintf(a,256,"%g",as_num(l));
389     if(r.t==V_STR) sprintf(b,256,"%s",r.as.str); else sprintf(b,256,"%g",as_num(r));
390     sprintf(buf,512,"%s%s",a,b); return V_str(buf);
391   }
392   double a=as_num(l),b=as_num(r);
393   if(strcmp(op,"+")==0) return V_num(a+b);
394   if(strcmp(op,"-")==0) return V_num(a-b);
395   if(strcmp(op,"*")==0) return V_num(a*b);
396   if(strcmp(op,"/")==0) return V_num(b!=0?a/b:0);
397   if(strcmp(op,"%")==0) return V_num(b!=0?fmod(a,b):0);
398   if(strcmp(op,"<")==0) return V_bool(a<b);
399   if(strcmp(op,"<=")==0) return V_bool(a<=b);
400   if(strcmp(op,">")==0) return V_bool(a>b);
401   if(strcmp(op,">=")==0) return V_bool(a>=b);
402   return V_nil();
403 }
404
405 static Value call_value(Value callee,Value*args,int n,Env*env){
406   if(callee.t==V_NATIVE) return callee.as.native(args,n,env);
407   if(callee.t==V_FUNC){
408     Node*decl=callee.as.fn.decl;
409     Env*local=env_new(callee.as.fn.closure);
410     for(int i=0;i<decl->nparams;i++)
411       env_set(local,decl->params[i], i<n?args[i]:V_nil());
412     Flow f=exec(decl->body,local);
413     return f.ret? f.val : V_nil();

```

```

414     }
415     if(callee.t==V_STRUCTURE){
416         /* construct instance: run body to collect default fields/methods */
417         Value inst; inst.t=V_INST; inst.as.inst.name=strdup(callee.as.st.name);
418         inst.as.inst.fields=env_new(callee.as.st.closure);
419         if(callee.as.st.decl&&callee.as.st.decl->body)
420             exec(callee.as.st.decl->body, inst.as.inst.fields);
421         return inst;
422     }
423     return V_nil();
424 }
425
426 static Value eval(Node*n, Env*env){
427     if(!n) return V_nil();
428     switch(n->type){
429         case N_INT: case N_FLOAT: return V_num(n->num);
430         case N_STRING: return V_str(n->str);
431         case N_BOOL: return V_bool(n->boolean);
432         case N_NIL: return V_nil();
433         case N_IDENT:{
434             if(strcmp(n->str,"STAR")==0) return V_star();
435             Value v; if(env_get(env,n->str,&v)) return v;
436             /* the ledger is addressable by several manuscript names */
437             if(strcmp(n->str,"TupleSpace")==0||strcmp(n->str,"tuplespace")==0||
438                strcmp(n->str,"Omega")==0){
439                 Value t; t.t=V_TUPLESPEACE; t.as.ts=G_LEDGER; return t;
440             }
441             /* unknown identifier resolves to its name as a symbol/string */
442             return V_str(n->str);
443         }
444         case N_ARRAY:{
445             Arr*a=arr_new();
446             for(int i=0;i<n->nkids;i++) arr_push(a,eval(n->kids[i],env));
447             return V_arr_take(a);
448         }
449         case N_UNARY:{
450             Value v=eval(n->kids[0],env);
451             if(strcmp(n->str,"-")==0) return V_num(-as_num(v));
452             if(strcmp(n->str,"!")==0) return V_bool(!truthy(v));
453             return v;
454         }
455         case N_BINARY:{
456             Value l=eval(n->kids[0],env), r=eval(n->kids[1],env);
457             return eval_binary(n->str,l,r);
458         }
459         case N_CONSIST:{
460             Value l=eval(n->kids[0],env), r=eval(n->kids[1],env);
461             return eval_binary("~",l,r);
462         }
463         case N_INDEX:{
464             Value base=eval(n->kids[0],env);
465             if(n->nkids<2) return V_nil();
466             Value idx=eval(n->kids[1],env);
467             int i=(int)as_num(idx);
468             if(base.t==V_ARR && i>=0 && i<base.as.arr->len) return base.as.arr->items[i];
469             if(base.t==V_DIST && i>=0 && i<base.as.dist->v) return V_num(base.as.dist->p[i]);
470             return V_nil();
471         }
472         case N_GET:{
473             Value base=eval(n->kids[0],env);
474             /* method/property access on builtins & instances */
475             if(base.t==V_INST){
476                 Value v; if(env_get(base.as.inst.fields,n->str,&v)) return v;
477             }
478             if(base.t==V_TUPLESPEACE){
479                 if(strcmp(n->str,"len")==0||strcmp(n->str,"facts")==0)
480                     return V_num(base.as.ts?base.as.ts->len:0);
481             }
482             if(base.t==V_DIST){
483                 if(strcmp(n->str,"entropy")==0){ Value a=base; return nf_entropy(&a,1,env); }
484                 if(strcmp(n->str,"v")==0||strcmp(n->str,"len")==0) return V_num(base.as.dist->v);
485             }
486             /* Namespace-style: Omega::DATABASE etc. -> return ledger object */
487             if(base.t==V_STR){
488                 if(strcmp(base.as.str,"Omega")==0||strcmp(n->str,"tuplespace")==0||
489                    strcmp(n->str,"DATABASE")==0)
490                     { Value v; v.t=V_TUPLESPEACE; v.as.ts=G_LEDGER; return v; }
491             }
492             /* member on string fallback: keep namespace path as string */
493             char buf[256]; snprintf(buf,256,"%s",n->str); return V_str(buf);
494         }
495         case N_CALL:{
496             Value callee=eval(n->kids[0],env);
497             int argc=n->nkids-1;

```

```

498     Value*args=argc>0?malloc(argc*sizeof(Value)):NULL;
499     for(int i=0;i<argc;i++) args[i]=eval(n->kids[i+1],env);
500     /* special: append on a tuplespace value */
501     if(n->kids[0]->type==N_GET){
502         Node*g=n->kids[0];
503         Value base=eval(g->kids[0],env);
504         if(base.t==V_TUPLESPEC && strcmp(g->str,"append")==0){
505             TupleSpace*ts=base.as.ts? base.as.ts : G_LEDGER;
506             if(ts){
507                 if(ts->len==ts->cap){ ts->cap=ts->cap?ts->cap*2:8;
508                     ts->facts=realloc(ts->facts,ts->cap*sizeof(Value)); }
509                 ts->facts[ts->len++]= argc>0?args[0]:V_nil();
510                 Value r=V_num(ts->len-1); free(args); return r;
511             }
512         }
513         if(base.t==V_TUPLESPEC && (strcmp(g->str,"read")==0||strcmp(g->str,"scan")==0)){
514             TupleSpace*ts=base.as.ts?base.as.ts:G_LEDGER;
515             Arr*out=arr_new();
516             if(ts) for(int i=0;i<ts->len;i++) arr_push(out,ts->facts[i]);
517             free(args); return V_arr_take(out);
518         }
519     }
520     Value r=call_value(callee,args,argc,env);
521     free(args);
522     return r;
523 }
524 case N_LAMBDA:{
525     Value v; v.t=V_FUNC; v.as.fn.decl=n; v.as.fn.closure=env; return v;
526 }
527 case N_EACH:{
528     Value coll=eval(n->kids[0],env);
529     Value fn=eval(n->kids[1],env); /* lambda */
530     if(coll.t==V_ARR && fn.t==V_FUNC){
531         for(int i=0;i<coll.as.arr->len;i++){
532             Value arg=coll.as.arr->items[i];
533             call_value(fn,&arg,1,env);
534         }
535     }
536     return V_nil();
537 }
538 case N_COMMIT:{
539     /* l >> r : commit. If r is a ledger append target, append l; else evaluate r. */
540     Value l=eval(n->kids[0],env);
541     Node*rn=n->kids[1];
542     /* pattern: X >> ledger.append(...) handled by call; otherwise treat as
543        "commit l, then yield r". We commit l to global ledger and eval r. */
544     if(G_LEDGER){
545         if(G_LEDGER->len==G_LEDGER->cap){ G_LEDGER->cap=G_LEDGER->cap?G_LEDGER->cap*2:8;
546             G_LEDGER->facts=realloc(G_LEDGER->facts,G_LEDGER->cap*sizeof(Value)); }
547         /* only commit substantive values (dist/inst/str), not nil */
548         if(l.t!=V_NIL) G_LEDGER->facts[G_LEDGER->len++]=l;
549     }
550     Value r=eval(rn,env);
551     return r.t!=V_NIL? r : l;
552 }
553 case N_QUERY:{
554     /* l => r : pattern query / map; we evaluate r in scope, returning it */
555     eval(n->kids[0],env);
556     return eval(n->kids[1],env);
557 }
558 case N_LARROW:{
559     /* facts <- fact : append element to an array-valued field */
560     Value tgt=eval(n->kids[0],env);
561     Value val=eval(n->kids[1],env);
562     if(tgt.t==V_ARR){ arr_push(tgt.as.arr,val); return tgt; }
563     return val;
564 }
565 default: break;
566 }
567 /* statements that can appear in expression position */
568 Flow f=exec(n,env);
569 return f.val;
570 }
571
572 static Flow exec(Node*n,Env*env){
573     if(!n) return flow_none();
574     switch(n->type){
575         case N_PROGRAM: case N_BLOCK:{
576             Flow f=flow_none();
577             for(int i=0;i<n->nkids;i++){
578                 f=exec(n->kids[i],env);
579                 if(f.ret) return f;
580             }
581             return f;

```

```

582     }
583     case N_BIND:{
584         Value v=eval(n->kids[0],env);
585         env_set(env,n->str,v);
586         Flow f=flow_none(); f.val=v; return f;
587     }
588     case N_ASSIGN:{
589         Node*lhs=n->kids[0];
590         Value v=eval(n->kids[1],env);
591         if(lhs->type==N_IDENT) env_set(env, lhs->str,v);
592         else if(lhs->type==N_INDEX){
593             Value base=eval(lhs->kids[0],env);
594             int i=(int)as_num(eval(lhs->kids[1],env));
595             if(base.t==V_ARR && i>=0 && i<base.as.arr->len) base.as.arr->items[i]=v;
596             else if(base.t==V_DIST && i>=0 && i<base.as.dist->p[i]) base.as.dist->p[i]=as_num(v);
597         }
598         Flow f=flow_none(); f.val=v; return f;
599     }
600     case N_FUNC:{
601         Value v; v.t=V_FUNC; v.as.fn.decl=n; v.as.fn.closure=env;
602         env_set(env,n->str?n->str:"anon",v);
603         Flow f=flow_none(); f.val=v; return f;
604     }
605     case N_STRUCT:{
606         Value v; v.t=V_STRUCT; v.as.st.name=strdup(n->str?n->str:"struct");
607         v.as.st.decl=n; v.as.st.closure=env;
608         env_set(env,n->str?n->str:"struct",v);
609         Flow f=flow_none(); f.val=v; return f;
610     }
611     case N_ASPERAL:{
612         /* union/gradual-type declaration: register name=STAR-capable marker */
613         Flow f=flow_none(); return f;
614     }
615     case N_OMEGA_DB:{
616         /* engine namespace block: execute its body in current env.
617            The ledger is the global G_LEDGER, exposed as 'tuplespace'/'Omega'. */
618         Value tsv; tsv.t=V_TUPLESAPCE; tsv.as.ts=G_LEDGER;
619         env_set(env,"tuplespace",tsv);
620         env_set(env,"Omega",tsv);
621         if(n->body) return exec(n->body,env);
622         return flow_none();
623     }
624     case N_REVISER:{
625         /* reflective learning loop: execute body with begin/ensure semantics.
626            Exceptions are modelled by checking for nil/failed results and
627            rolling back (we simply run the body; faulty rewrites no-op). */
628         if(n->body) return exec(n->body,env);
629         return flow_none();
630     }
631     case N_IF:{
632         Value c=eval(n->kids[0],env);
633         if(truthy(c)) return exec(n->kids[1],env);
634         else if(n->nkids>2) return exec(n->kids[2],env);
635         return flow_none();
636     }
637     case N_WHILE:{
638         while(truthy(eval(n->kids[0],env))){
639             Flow f=exec(n->kids[1],env);
640             if(f.ret) return f;
641         }
642         return flow_none();
643     }
644     case N_FOR:{
645         Value it=eval(n->kids[0],env);
646         Env*loop=env_new(env);
647         if(it.t==V_ARR){
648             for(int i=0;i<it.as.arr->len;i++){
649                 env_set(loop,n->str,it.as.arr->items[i]);
650                 Flow f=exec(n->kids[1],loop);
651                 if(f.ret) return f;
652             }
653         } else if(it.t==V_DIST){
654             for(int i=0;i<it.as.dist->p[i];i++){
655                 env_set(loop,n->str,V_num(it.as.dist->p[i]));
656                 Flow f=exec(n->kids[1],loop);
657                 if(f.ret) return f;
658             }
659         }
660         return flow_none();
661     }
662     case N_RETURN:{
663         Flow f; f.ret=true; f.val= n->nkids? eval(n->kids[0],env):V_nil(); return f;
664     }
665     case N_PRINT:{

```

```
666         for(int i=0;i<n->nkids;i++){ if(i)printf(" "); print_value(eval(n->kids[i],env)); }
667         printf("\n");
668         return flow_none();
669     }
670     default:{
671         Flow f=flow_none(); f.val=eval(n,env); return f;
672     }
673 }
674 }
675
676 int interp_run(Node*program){
677     G_LEDGER=calloc(1,sizeof(TupleSpace));
678     Env*g=env_new(NULL);
679     install_natives(g);
680     /* expose ledger as Omega / tuplespace from the start */
681     Value tsv; tsv.t=V_TUPLESPEC; tsv.as.ts=G_LEDGER;
682     env_set(g,"Omega",tsv); env_set(g,"tuplespace",tsv);
683     exec(program,g);
684     return 0;
685 }
```

src/transpile.c

Bada -> C compiler (transpiler)

```

1 #define _GNU_SOURCE
2 /* transpile.c -- Bada -> C compiler.
3 *
4 * Strategy: emit a self-contained C program that embeds a small runtime
5 * (a tagged Value union, a dist/phase model, the append-only TupleSpace, and
6 * the engine builtins), then lowers the Bada AST into calls against that
7 * runtime. The emitted C is compiled by gcc into a native executable.
8 *
9 * This is a faithful "Bada compiler": the same programs the interpreter runs
10 * compile to C and produce identical numerics (zero preservation; |psi|^2==a).
11 *
12 * To keep the lowering tractable, the transpiler targets the engine-shaped
13 * subset actually used by the manuscript driver: bindings (:=), calls to the
14 * native builtins, print, arrays, dist literals, and the commit (>) operator.
15 * Constructs outside this subset are emitted as comments so the C still builds.
16 */
17 #include "bada.h"
18
19 typedef struct { FILE*f; int tmp; } Emit;
20
21 static const char *RUNTIME =
22 "/* ==== Bada C runtime (emitted) ==== */\n"
23 "#include <stdio.h>\n#include <stdlib.h>\n#include <string.h>\n#include <math.h>\n#include <complex.h>\n"
24 "typedef enum {BV_NIL, BV_NUM, BV_STR, BV_ARR, BV_DIST} BVT;\n"
25 "typedef struct BV BV;\n"
26 "typedef struct {BV*it; int n, cap;} BArr;\n"
27 "typedef struct {double*p; int v;} BDist;\n"
28 "struct BV {BVT t; double num; char*str; BArr*arr; BDist*dist;};\n"
29 "static BV bnul() {BV v; v.t=BV_NIL; return v;}\n"
30 "static BV bnum(double x) {BV v; v.t=BV_NUM; v.num=x; return v;}\n"
31 "static BV bstr(const char*s) {BV v; v.t=BV_STR; v.str=strdup(s); return v;}\n"
32 "static BArr*barr() {BArr*a=malloc(1, sizeof(BArr)); return a;}\n"
33 "static void bpush(BArr*a, BV v) {if(a->n==a->cap) {a->cap=a->cap*2; a->it=realloc(a->it, a->cap*sizeof(BV)); a->n++;} a->it[a->n]=v;}\n"
34 "static BV barrv(BArr*a) {BV v; v.t=BV_ARR; v.arr=a; return v;}\n"
35 "static BDist*bdist(int v) {BDist*d=malloc(sizeof(BDist)); d->p=malloc(v, sizeof(double)); d->v=v; return d;}\n"
36 "static BV bdistv(BDist*d) {BV v; v.t=BV_DIST; v.dist=d; return v;}\n"
37 "static double bnumof(BV v) {return v.t==BV_NUM?v.num:0;}\n"
38 "/* append-only ledger */\n"
39 "static BV*G_FACTS=0; static int G_N=0, G_CAP=0;\n"
40 "static int bcommit(BV v) {if(G_N>=G_CAP) {G_CAP=G_CAP*2; G_FACTS=realloc(G_FACTS, G_CAP*sizeof(BV));} G_FACTS[G_N]=v; return G_N++;}\n"
41 "static int bledger_len() {return G_N;}\n"
42 "/* engine builtins */\n"
43 "static BV b_softmax(BV a) {int v; double*src; int own=0;\n"
44 "  if(a.t==BV_DIST) {v=a.dist->p; src=a.dist->p; } else if(a.t==BV_ARR) {v=a.arr->n; src=malloc(v*sizeof(double)); for(int i=0; i<v; i++) src[i]=bnumof(a.arr->it[i]); own=1; } else return bnul();\n"
45 "  BDist*o=bdist(v); double mx=-INFINITY; for(int i=0; i<v; i++) if(src[i]>mx) mx=src[i]; double s=0; for(int i=0; i<v; i++) {o->p[i]=exp(src[i]-mx); s+=o->p[i]; } for(int i=0; i<v; i++) o->p[i]/=s; for(int i=0; i<v; i++) if(src[i]-mx<-30.0) o->p[i]=0.0; {double ss=0; for(int i=0; i<v; i++) ss+=o->p[i]; if(ss>0) for(int i=0; i<v; i++) o->p[i]/=ss; if(own) free(src); return bdistv(o); }\n"
46 "static BV b_entropy(BV a) {double*p; int v; int own=0; if(a.t==BV_DIST) {p=a.dist->p; v=a.dist->v; } else if(a.t==BV_ARR) {v=a.arr->n; p=malloc(v*sizeof(double)); for(int i=0; i<v; i++) p[i]=bnumof(a.arr->it[i]); own=1; } else return bnum(0);\n"
47 "  double H=0; for(int i=0; i<v; i++) {double q=p[i]; if(q<1e-12) q=1e-12; H+=-q*log(q); if(own) free(p); return bnum(H); }\n"
48 "static BV b_unknown_prior(BV a) {int v=(int)bnumof(a); if(v<1) v=1; BDist*d=bdist(v); for(int i=0; i<v; i++) d->p[i]=1.0/v; return bdistv(d); }\n"
49 "static BV b_update(BV pr, BV ev, double lr) {if(pr.t!=BV_DIST) return pr; BDist*P=pr.dist; double*E; int ev_v; int own=0;\n"
50 "  if(ev.t==BV_DIST) {E=ev.dist->p; ev_v=ev.dist->v; } else if(ev.t==BV_ARR) {ev_v=ev.arr->n; E=malloc(ev_v*sizeof(double)); for(int i=0; i<ev_v; i++) E[i]=bnumof(ev.arr->it[i]); own=1; } else return pr;\n"
51 "  int v=P->v; BDist*o=bdist(v); double s=0; for(int i=0; i<v; i++) {double e=i<ev_v?E[i]:0; o->p[i]=(1-lr)*P->p[i]+lr*e; s+=o->p[i]; } if(s>0) for(int i=0; i<v; i++) o->p[i]/=s; if(own) free(E); return bdistv(o); }\n"
52 "static BV b_cognitive(BV base, BArr*mods, BArr*betas) {BDist*a;\n"
53 "  if(base.t==BV_DIST) {a=bdist(base.dist->p); for(int i=0; i<a->v; i++) a->p[i]=base.dist->p[i]; } else {BV s=b_softmax(base); a=bdist(s.dist->p); for(int i=0; i<a->v; i++) a->p[i]=s.dist->p[i]; }\n"
54 "  int v=a->v; double*th=malloc(v, sizeof(double));\n"
55 "  if(mods) for(int m=0; m<mods->n; m++) {if(mods->it[m].t!=BV_ARR) continue; BArr*pr=mods->it[m].arr; if(pr->n<2) continue;\n"
56 "    double*q; int qv; int own=0; if(pr->it[0].t==BV_DIST) {q=pr->it[0].dist->p; qv=pr->it[0].dist->v; } else if(pr->it[0].t==BV_ARR) {qv=pr->it[0].arr->n; q=malloc(qv*sizeof(double)); for(int i=0; i<qv; i++) q[i]=bnumof(pr->it[0].arr->it[i]); own=1; } else continue;\n"
57 "    double H=0; for(int i=0; i<qv; i++) {double qq=q[i]; if(qq<1e-12) qq=1e-12; H+=-qq*log(qq); }\n"
58 "    BArr*phi=pr->it[1].t==BV_ARR?pr->it[1].arr:0; double beta=(betas&&mods->n)?bnumof(betas->it[0]):1.0;\n"
59 "    for(int i=0; i<v; i++) {double pv=(phi&&i<phi->n)?bnumof(phi->it[i]):0; th[i]=beta*H*pv; if(own) free(q); }\n"
60 "    BDist*qh=bdist(v); double s=0; for(int i=0; i<v; i++) {double complex psi=sqrt(a->p[i])*cexp(I*th[i]); double mag2=creal(psi)*creal(psi)+cimag(psi)*cimag(psi); qh->p[i]=mag2; s+=mag2; } if(s>0) for(int i=0; i<v; i++) qh->p[i]/=s; free(th);\n"
61 "    BDist*snap=bdist(v); for(int i=0; i<v; i++) snap->p[i]=qh->p[i]; bcommit(bdistv(snap)); return bdistv(qh); }\n"
62 "static void bprint(BV v) {switch(v.t) {case BV_NIL: printf("\nil\n"); break; case BV_NUM: if(v.num==(long long)v.num) print f("\%lld\n", (long long)v.num); else printf("\%.5f\n", v.num); break; case BV_STR: printf("\%s\n", v.str); break; case BV_ARR: {p

```

```

100 printf("\n");for(int i=0;i<v.arr->n;i++){if(i)printf("\", ");bprint(v.arr->it[i]);}printf("\n");break;}case BV_DIST
101 :{printf("\ndist[");for(int i=0;i<v.dist->v;i++){if(i)printf("\", ");printf("%.5f",v.dist->p[i]);}printf("\n");br
102 eak;}}}\n"
103 "static BV badd(BV a,BV b){return bnum(bnumof(a)+bnumof(b));}\n"
104 "static BV bsub(BV a,BV b){return bnum(bnumof(a)-bnumof(b));}\n"
105 "static BV bmul(BV a,BV b){return bnum(bnumof(a)*bnumof(b));}\n"
106 "static BV bdiv(BV a,BV b){double d=bnumof(b);return bnum(d!=0?bnumof(a)/d:0);}\n";
107
108 /* ---- expression lowering: returns a C expression string (heap) ---- */
109 static char *lower_expr(Emit*E,Node*n);
110
111 static char *sdup(const char*s){ return strdup(s); }
112 static char *fmt(const char*f,...){
113     char buf[4096]; va_list ap; va_start(ap,f); vsnprintf(buf,sizeof buf,f,ap); va_end(ap);
114     return strdup(buf);
115 }
116
117 static char *lower_call(Emit*E,Node*n){
118     Node*callee=n->kids[0];
119     int argc=n->nkids-1;
120     /* builtins by name */
121     if(callee->type==N_IDENT){
122         const char*name=callee->str;
123         char*a0=argc>0?lower_expr(E,n->kids[1]):sdup("bnil()");
124         if(strcmp(name,"softmax")==0){ char*r=fmt("b_softmax(%)",a0); free(a0); return r; }
125         if(strcmp(name,"entropy")==0){ char*r=fmt("b_entropy(%)",a0); free(a0); return r; }
126         if(strcmp(name,"unknown_prior")==0){ char*r=fmt("b_unknown_prior(%)",a0); free(a0); return r; }
127         if(strcmp(name,"dist")==0){ char*r=fmt("(%)",a0); free(a0); return r; }
128         if(strcmp(name,"len")==0){ char*r=fmt("bnum((%).t==BV_ARR?(%).arr->n:((%).t==BV_DIST?(%).dist->v:0))",a0
129 ,a0,a0,a0); free(a0); return r; }
130         if(strcmp(name,"sqrt")==0){ char*r=fmt("bnum(sqrt(bnumof(%.t==BV_ARR?(%).arr->n:((%).t==BV_DIST?(%).dist->v:0)))",a0); free(a0); return r; }
131         if(strcmp(name,"log")==0){ char*r=fmt("bnum(log(bnumof(%.t==BV_ARR?(%).arr->n:((%).t==BV_DIST?(%).dist->v:0)))",a0); free(a0); return r; }
132         if(strcmp(name,"exp")==0){ char*r=fmt("bnum(exp(bnumof(%.t==BV_ARR?(%).arr->n:((%).t==BV_DIST?(%).dist->v:0)))",a0); free(a0); return r; }
133         if(strcmp(name,"ledger")==0){ free(a0); return sdup("bnum(ledger_len())"); }
134         if(strcmp(name,"update")==0){
135             char*a1=argc>1?lower_expr(E,n->kids[2]):sdup("bnil()");
136             char*a2=argc>2?lower_expr(E,n->kids[3]):NULL;
137             char*r;
138             if(a2){ r=fmt("b_update(%,%,bnumof(%.t==BV_ARR?(%).arr->n:((%).t==BV_DIST?(%).dist->v:0))",a0,a1,a2); free(a2); }
139             else r=fmt("b_update(%,%,0.5)",a0,a1);
140             free(a0); free(a1); return r;
141         }
142         if(strcmp(name,"cognitive_system")==0){
143             char*a1=argc>1?lower_expr(E,n->kids[2]):sdup("0");
144             char*a2=argc>2?lower_expr(E,n->kids[3]):sdup("0");
145             /* modules/betas must be BArr*: wrap if they are array values */
146             char*r=fmt("b_cognitive(%,(%.t==BV_ARR?(%).arr:0,(%.t==BV_ARR?(%).arr:0)",
147 a0,a1,a1,a2,a2);
148             free(a0);free(a1);free(a2); return r;
149         }
150     }
151     free(a0);
152 }
153
154 /* method call: x.append(v) -> bcommit(v) */
155 if(callee->type==N_GET && strcmp(callee->str,"append")==0){
156     char*a0=argc>0?lower_expr(E,n->kids[1]):sdup("bnil()");
157     char*r=fmt("bnum(bcommit(%.t==BV_ARR?(%).arr:0,(%.t==BV_ARR?(%).arr:0))",a0,a0); free(a0); return r;
158 }
159
160 /* fallback: nil */
161 return sdup("bnil()");
162 }
163
164 static char *lower_expr(Emit*E,Node*n){
165     if(!n) return sdup("bnil()");
166     switch(n->type){
167         case N_INT: case N_FLOAT: return fmt("bnum(%.17g)",n->num);
168         case N_STRING: return fmt("bstr(\"%s\")",n->str?n->str:"");
169         case N_BOOL: return fmt("bnum(%d)",n->boolean?1:0);
170         case N_NIL: return sdup("bnil()");
171         case N_IDENT: {
172             /* identifiers map to local C variables prefixed v_ */
173             return fmt("v_%s",n->str);
174         }
175         case N_ARRAY: {
176             /* build a temp array */
177             int id=E->tmp++;
178             fprintf(E->f," BArr*t%d=barr();\n",id);
179             for(int i=0;i<n->nkids;i++){
180                 char*e=lower_expr(E,n->kids[i]);
181                 fprintf(E->f," bpush(t%d,%s);\n",id,e); free(e);
182             }
183             return fmt("barrv(t%d)",id);
184         }
185         case N_BINARY: {
186             char*l=lower_expr(E,n->kids[0]);

```

```

143     char*r=lower_expr(E,n->kids[1]);
144     const char*op=n->str; char*res;
145     if(strcmp(op,"+")==0) res=fmt("badd(%s,%s)",l,r);
146     else if(strcmp(op,"-")==0) res=fmt("bsub(%s,%s)",l,r);
147     else if(strcmp(op,"*")==0) res=fmt("bmul(%s,%s)",l,r);
148     else if(strcmp(op,"/")==0) res=fmt("bdiv(%s,%s)",l,r);
149     else res=fmt("bnum(bnumof(%s)%sbnunof(%s))",l,
150               strcmp(op,"<")==0?"<":strcmp(op,">")==0?">":"+ ",r);
151     free(l);free(r); return res;
152 }
153 case N_CALL: return lower_call(E,n);
154 case N_UNARY:{
155     char*e=lower_expr(E,n->kids[0]);
156     char*r;
157     if(strcmp(n->str,"-")==0) r=fmt("bnum(-bnumof(%s))",e);
158     else if(strcmp(n->str,"!")==0) r=fmt("bnum(bnumof(%s)==0?1:0)",e);
159     else r=fmt("(%s)",e);
160     free(e); return r;
161 }
162 case N_INDEX:{
163     char*b=lower_expr(E,n->kids[0]);
164     char*i=n->nkids>1?lower_expr(E,n->kids[1]):sdup("bnum(0)");
165     char*r=fmt("((%s).t==BV_DIST?bnum((%s).dist->p[(int)bnumof(%s)]):((%s).t==BV_ARR?(%s).arr->it[(int)bnumof(%s)]):bnil())",
166             b,b,i,b,b,i);
167     free(b);free(i); return r;
168 }
169 case N_COMMIT:{
170     char*l=lower_expr(E,n->kids[0]);
171     int id=E->tmp++;
172     fprintf(E->f," BV c%d=%s; bcommit(c%d);\n",id,l,id); free(l);
173     char*r=lower_expr(E,n->kids[1]);
174     return r;
175 }
176 case N_GET: return sdup("bnil()");
177 default: return sdup("bnil()");
178 }
179 }
180
181 static void lower_stmt(Emit*E,Node*n){
182     switch(n->type){
183     case N_BIND:{
184         char*e=lower_expr(E,n->kids[0]);
185         fprintf(E->f," BV v_%s=%s; (void)v_%s;\n",n->str,e,n->str); free(e);
186         break;
187     }
188     case N_ASSIGN:{
189         if(n->kids[0]->type==N_IDENT){
190             char*e=lower_expr(E,n->kids[1]);
191             fprintf(E->f," v_%s=%s;\n",n->kids[0]->str,e); free(e);
192         }
193         break;
194     }
195     case N_PRINT:{
196         for(int i=0;i<n->nkids;i++){
197             char*e=lower_expr(E,n->kids[i]);
198             fprintf(E->f," bprint(%s);",e); free(e);
199             if(i<n->nkids-1) fprintf(E->f,"printf(\" \");");
200         }
201         fprintf(E->f,"printf(\"\\n\\n\");\n");
202         break;
203     }
204     case N_OMEGA_DB: case N_REVISER:
205         if(n->body) for(int i=0;i<n->body->nkids;i++) lower_stmt(E,n->body->kids[i]);
206         break;
207     case N_BLOCK:
208         for(int i=0;i<n->nkids;i++) lower_stmt(E,n->kids[i]);
209         break;
210     case N_FUNC: case N_STRUCT: case N_ASPEARL:
211         fprintf(E->f," /* decl '%s' specialised at compile time (interp register) */\n",
212             n->str?n->str:"?");
213         break;
214     case N_IF:{
215         char*c=lower_expr(E,n->kids[0]);
216         fprintf(E->f," if(bnumof(%s)!=0){\n",c); free(c);
217         lower_stmt(E,n->kids[1]);
218         fprintf(E->f," }");
219         if(n->nkids>2){ fprintf(E->f," else {\n"); lower_stmt(E,n->kids[2]); fprintf(E->f," }"); }
220         fprintf(E->f,"");
221         break;
222     }
223     default:{
224         char*e=lower_expr(E,n);
225         fprintf(E->f," (void)(%s);\n",e); free(e);

```

```
226     }
227 }
228 }
229
230 int transpile_to_c(Node*program, const char*out_path){
231     FILE*f=fopen(out_path, "w");
232     if(!f){ perror("open output"); return 1; }
233     Emit E={f, 0};
234     fputs(RUNTIME, f);
235     fputs("\nint main(void){\n", f);
236     for(int i=0; i<program->nkids; i++) lower_stmt(&E, program->kids[i]);
237     fputs("    return 0;\n}\n", f);
238     fclose(f);
239     return 0;
240 }
```

src/main.c

CLI driver

```

1  #define _GNU_SOURCE
2  /* main.c -- the `bada` command-line driver. */
3  #include "bada.h"
4
5  static char *read_file(const char*path){
6      FILE*f=fopen(path,"rb");
7      if(!f){ fprintf(stderr,"cannot open %s\n",path); return NULL; }
8      fseek(f,0,SEEK_END); long sz=ftell(f); fseek(f,0,SEEK_SET);
9      char*buf=malloc(sz+1); fread(buf,1,sz,f); buf[sz]=0; fclose(f);
10     return buf;
11 }
12
13 static void usage(const char*p){
14     fprintf(stderr,
15         "Bada -- the Unknown-Prior Engine language\n"
16         "usage:\n"
17         "  %s run    <file.bada>      # interpret\n"
18         "  %s build  <file.bada> [-o out] # compile to C and gcc-build a binary\n"
19         "  %s emit   <file.bada> [-o c]  # emit C only\n"
20         "  %s tokens <file.bada>      # dump token stream\n"
21         "  %s ast    <file.bada>      # dump AST\n", p,p,p,p,p);
22 }
23
24 static void dump_tokens(const char*src){
25     Lexer lx; lexer_init(&lx,src);
26     for(;;){ Token t=lexer_next(&lx);
27         printf("%4d %-10s %s\n",t.line,toktype_name(t.type),t.lexeme?t.lexeme:"");
28         TokenType ty=t.type; token_free(&t);
29         if(ty==T_EOF) break; }
30 }
31
32 static const char*nodename(NodeType t){
33     switch(t){case N_PROGRAM:return"program";case N_BIND:return"bind";case N_FUNC:return"func";
34         case N_STRUCT:return"struct";case N_OMEGA_DB:return"omega_db";case N_REVISER:return"reviser";
35         case N_CALL:return"call";case N_BINARY:return"binary";case N_IDENT:return"ident";
36         case N_INT:return"int";case N_FLOAT:return"float";case N_STRING:return"string";
37         case N_ARRAY:return"array";case N_PRINT:return"print";case N_COMMIT:return"commit";
38         case N_QUERY:return"query";case N_BLOCK:return"block";case N_RETURN:return"return";
39         case N_LAMBDA:return"lambda";case N_EACH:return"each";case N_INDEX:return"index";
40         case N_GET:return"get";case N_IF:return"if";case N_FOR:return"for";case N_ASSIGN:return"assign";
41         case N_ASPERAL:return"asperal";default:return"node";}
42 }
43
44 static void dump_ast(Node*n,int d){
45     for(int i=0;i<d;i++) printf(" ");
46     printf("%s",nodename(n->type));
47     if(n->str) printf(" '%s'",n->str);
48     if(n->type==N_INT||n->type==N_FLOAT) printf(" %g",n->num);
49     if(n->nparams){ printf(" |"); for(int i=0;i<n->nparams;i++) printf("%s%s",i?" ":"",n->params[i]); printf("|"); }
50     printf("\n");
51     for(int i=0;i<n->nkids;i++) dump_ast(n->kids[i],d+1);
52     if(n->body) dump_ast(n->body,d+1);
53 }
54
55 int main(int argc,char**argv){
56     if(argc<3){ usage(argv[0]); return 1; }
57     const char*cmd=argv[1];
58     const char*path=argv[2];
59     const char*out=NULL;
60     for(int i=3;i<argc-1;i++) if(strcmp(argv[i],"-o")==0) out=argv[i+1];
61
62     char*src=read_file(path);
63     if(!src) return 1;
64
65     if(strcmp(cmd,"tokens")==0){ dump_tokens(src); free(src); return 0; }
66
67     Node*prog=parse_program(src);
68
69     if(strcmp(cmd,"ast")==0){ dump_ast(prog,0); }
70     else if(strcmp(cmd,"run")==0){ interp_run(prog); }
71     else if(strcmp(cmd,"emit")==0){
72         const char*o=out?out:"out.c";
73         transpile_to_c(prog,o);
74         fprintf(stderr,"emitted C -> %s\n",o);
75     }
76     else if(strcmp(cmd,"build")==0){
77         const char*o=out?out:"a.out";
78         char cfile[512]; snprintf(cfile,sizeof cfile,"%s.gen.c",o);
79         transpile_to_c(prog,cfile);
80         char cmdline[1024];

```

```
80     snprintf(cmdline,sizeof cmdline,"gcc -O2 -o %s %s -lm",o,cfile);
81     fprintf(stderr,"[bada build] %s\n",cmdline);
82     int rc=system(cmdline);
83     if(rc==0) fprintf(stderr,"[bada build] ok -> %s\n",o);
84     else fprintf(stderr,"[bada build] gcc failed (rc=%d)\n",rc);
85 }
86 else { usage(argv[0]); }
87
88 node_free(prog);
89 free(src);
90 return 0;
91 }
```

examples/hello.bada

Example: language sanity check

```
1 # hello.bada -- basic sanity check of the Bada language.
2
3 print("Bada online.")
4
5 # native dist type via max-entropy prior
6 prior := unknown_prior(4)
7 print("uniform prior:", prior)
8 print("entropy(uniform) =", entropy(prior))
9
10 # softmax over logits -> dist
11 a := softmax([2.0, 1.0, 0.0, -1.0])
12 print("softmax:", a)
13
14 # sharpen the prior with evidence; entropy must drop
15 post := update(prior, a)
16 print("post:", post)
17 print("entropy(post) =", entropy(post))
18
19 # the append-only ledger
20 prior >> tuplespace
21 print("ledger facts:", len(tuplespace))
```

examples/engine.bada

Example: full Unknown-Prior Engine

```

1 # =====
2 # engine.bada -- The Unknown-Prior Engine, written in Bada.
3 #
4 # A direct transcription of the manuscript's Modules 1-7 into the
5 # Bada language (this implementation), reproducing the two provable
6 # properties on V = 8 with tokens 3 and 6 ruled out:
7 # (i) confident zeros are preserved exactly
8 # (ii) |psi|^2 == base a (phase only reweights, never destroys)
9 # and the unknown-prior sharpening (entropy strictly decreases).
10 # =====
11
12 print("=====")
13 print("UNKNOWN-PRIOR ENGINE -- Bada reference run")
14 print("=====")
15
16 # ---- Module 4: the base distribution a = softmax(z) ----
17 # Logits chosen so that tokens 3 and 6 are ruled out (-inf -> p = 0).
18 # Bada uses a very large negative logit to denote a confident zero.
19 z := [1.6, 0.6, 0.1, -40.0, -0.4, 1.1, -40.0, 0.4]
20 a := softmax(z)
21
22 V := len(a)
23 print("Vocabulary size V =", V)
24
25 # ---- Module 5: the entropy-phase cognitive system (core) ----
26 # Two informative modules (retrieval, syntax) plus one uninformed module.
27 # Each module is a pair [q_m, phi_m]; betas are the gains.
28 retrieval := [ [0.30, 0.20, 0.10, 0.00, 0.15, 0.20, 0.00, 0.05],
29               [0.10, 0.20, 0.30, 0.00, 0.05, 0.25, 0.00, 0.05] ]
30 syntax    := [ [0.05, 0.25, 0.25, 0.00, 0.10, 0.25, 0.00, 0.10],
31               [0.20, 0.10, 0.10, 0.00, 0.20, 0.30, 0.00, 0.10] ]
32 uninformed := [ [0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125, 0.125],
33               [0.50, 0.50, 0.50, 0.50, 0.50, 0.50, 0.50, 0.50] ]
34
35 modules := [ retrieval, syntax, uninformed ]
36 betas   := [ 0.7, 0.5, 0.9 ]
37
38 # run the core: psi = sqrt(a) exp(i theta); q_hat = |psi|^2 / sum
39 q_hat := cognitive_system(a, modules, betas)
40
41 print("Ledger facts after inference:", len(tuplespace), "(append-only)")
42 print("")
43 print("token : base a_i      posterior q_hat_i")
44 print("-----")
45 for i in z {
46     # i ranges over logits' values via dist iteration; recompute index below
47 }
48
49 # explicit per-token print (indexing into the native dist)
50 print("0 :", f5(a[0]), f5(q_hat[0]))
51 print("1 :", f5(a[1]), f5(q_hat[1]))
52 print("2 :", f5(a[2]), f5(q_hat[2]))
53 print("3 :", f5(a[3]), f5(q_hat[3]), " <-- ruled out")
54 print("4 :", f5(a[4]), f5(q_hat[4]))
55 print("5 :", f5(a[5]), f5(q_hat[5]))
56 print("6 :", f5(a[6]), f5(q_hat[6]), " <-- ruled out")
57 print("7 :", f5(a[7]), f5(q_hat[7]))
58
59 # ---- Property (i): zero preservation ----
60 print("")
61 print("Property (i) zero preservation:")
62 print("  base zeros    =", zeros_of(a))
63 print("  posterior zeros =", zeros_of(q_hat))
64
65 # ---- Property (ii): |psi|^2 == base a (max |diff|) ----
66 print("")
67 print("Property (ii) |psi|^2 == base a (max |diff|):")
68 print("  ", sci(maxdiff(q_hat, last_a())), "(machine-zero => phase only reweights)")
69
70 # ---- Module 6: the @reviser learning loop ----
71 # Unknown prior starts uniform (max entropy) and is sharpened by evidence.
72 print("")
73 print("=====")
74 print("Unknown-prior sharpening (entropy decreases as evidence arrives)")
75 print("=====")
76
77 prior := unknown_prior(V)
78 print("step 0 (uniform prior) entropy =", entropy(prior))
79

```

```
80 @reviser {
81     prior := update(prior, q_hat)
82     prior >> tuplespace
83 }
84 print("step 1 (after evidence) entropy =", entropy(prior))
85
86 @reviser {
87     prior := update(prior, q_hat)
88     prior >> tuplespace
89 }
90 print("step 2 (more evidence) entropy =", entropy(prior))
91
92 print("")
93 print("(entropy strictly decreases: the engine earns its certainty)")
94 print("=====")
```

examples/core.bada

Example: engine core for the compiler path

```
1 # core.bada -- the engine core, in the transpiler-targeted Bada subset.
2 # This file is meant for `bada build` (Bada -> C -> native binary).
3
4 z := [1.6, 0.6, 0.1, -40.0, -0.4, 1.1, -40.0, 0.4]
5 a := softmax(z)
6
7 retrieval := [ [0.30,0.20,0.10,0.00,0.15,0.20,0.00,0.05],
8               [0.10,0.20,0.30,0.00,0.05,0.25,0.00,0.05] ]
9 syntax     := [ [0.05,0.25,0.25,0.00,0.10,0.25,0.00,0.10],
10               [0.20,0.10,0.10,0.00,0.20,0.30,0.00,0.10] ]
11 uninformed := [ [0.125,0.125,0.125,0.125,0.125,0.125,0.125,0.125],
12               [0.50,0.50,0.50,0.50,0.50,0.50,0.50,0.50] ]
13 modules := [ retrieval, syntax, uninformed ]
14 betas    := [ 0.7, 0.5, 0.9 ]
15
16 q_hat := cognitive_system(a, modules, betas)
17
18 print("compiled engine: base a    =", a)
19 print("compiled engine: q_hat     =", q_hat)
20 print("compiled engine: V         =", len(a))
21 print("compiled engine: a[3]      =", a[3])
22 print("compiled engine: a[6]      =", a[6])
23 print("compiled engine: entropy   =", entropy(q_hat))
```